CS6008 Assignment 2 — DNS Security

Overview

This report covers CS6008 Assignment 2 (DNS Security), built and tested as seven progressively hardened Docker Compose deployments. Each phase reuses the previous phase's topology and adds one new control, one new attack, or both, so the arc of the assignment is a single narrative: plain DNS has no origin authentication at all (Phase 1 baseline, Phase 2 on-path forgery) → DNSSEC fixes that for the resolver (Phase 3) → but only if the chain of trust is actually intact end to end, which an attacker cannot forge around without the parent's private key (Phase 4: signature stripping and the DS-omission downgrade) → denial-of-existence proofs, if naively signed, leak the zone's contents even though they fix the original gap (Phase 5: NSEC zone walking vs. NSEC3) → DNSSEC's guarantee stops at the resolver, not the stub client, so the last mile is still wide open (Phase 6: last-mile forgery past an already-validating resolver) → encrypting and authenticating that last mile closes the forgery, but only for a client that refuses to fall back to cleartext when the encrypted channel fails (Phase 7: DNS-over-TLS, and why a fallback policy undoes everything).

All seven phases run on the same three-subnet Docker topology (`client_net` 172.16.1.0/24, `resolver_net` 172.16.2.0/24, `auth_net` 172.16.3.0/24), each network `internal: true` so cross-subnet reachability exists only through the static routes each container's entrypoint installs — isolation and on-path positioning are both true by construction, not by firewall rules bolted on afterward. BIND 9.18 serves every authoritative and resolver role; every attack tool is a C/C++ program built directly on raw sockets and libpcap (`pcap_open_live`/`pcap_compile`/`pcap_inject`/`pcap_loop`), per the assignment's tooling constraint — no Scapy, no dnspython. The one deliberate exception is Phase 7's impersonation tool, which uses OpenSSL and ordinary POSIX sockets instead of libpcap, because completing a cryptographically valid TLS handshake requires a real TLS stack negotiating real key material; that program's own header comment explains the reasoning in full.

Every phase below follows the same shape: what changed from the previous phase, how it was verified as healthy before any attack was mounted (never blame an attack result on a chain that was already broken), the attack itself with evidence pulled from this repository's `screenshots/` directories, and the phase's own report questions answered directly. A closing appendix documents how Claude was used throughout.

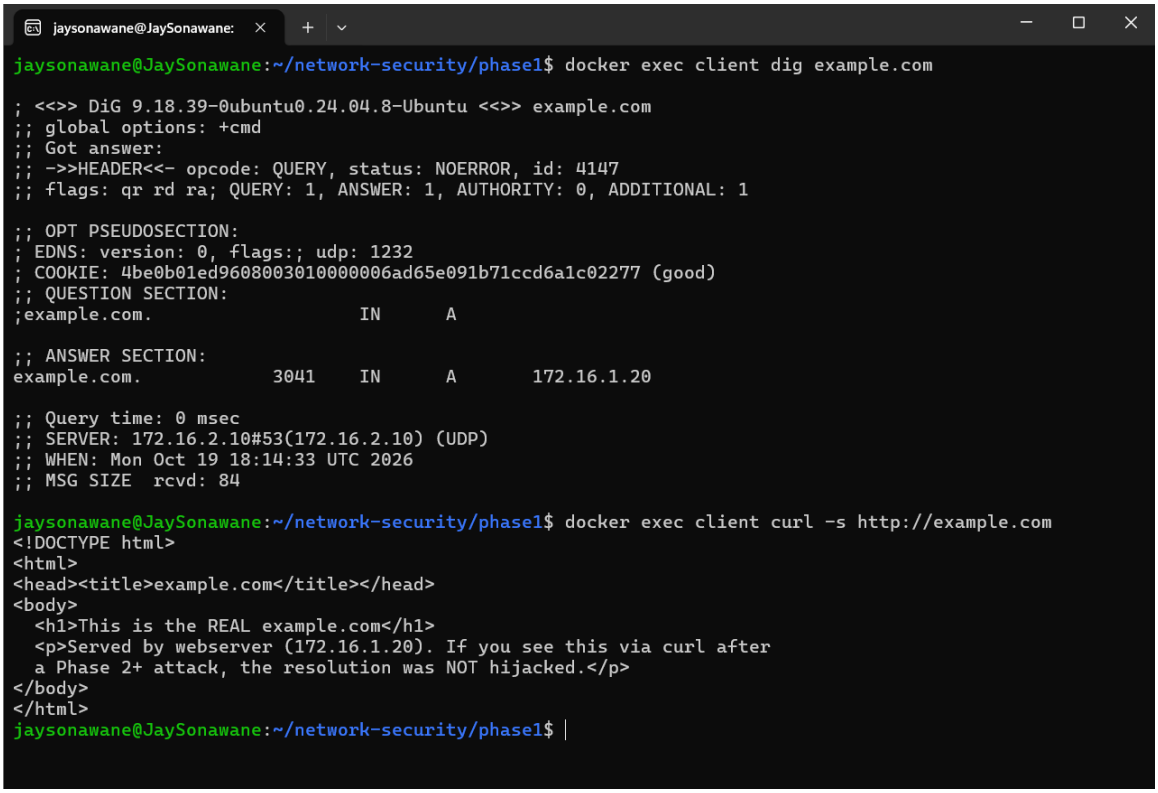
Phase 1 — Building the DNS Hierarchy

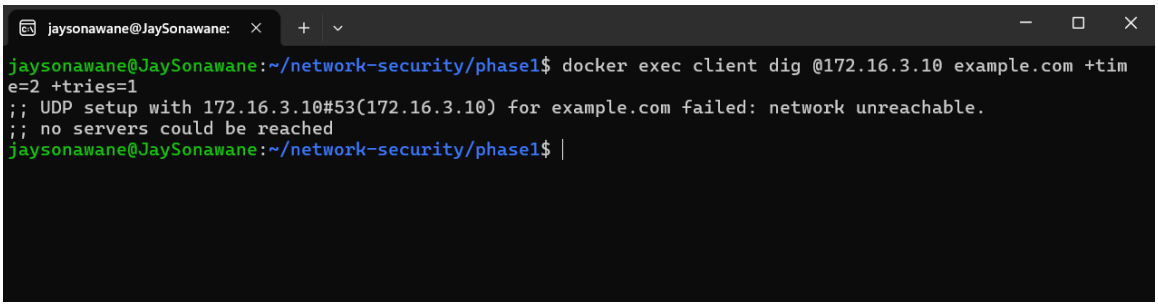
A three-subnet topology with full root → `.com` → `example.com` delegation (root-ns → tld-ns → domain-ns), a recursive resolver with DNSSEC validation explicitly off, and a plain client/webserver pair. No attacker is present yet — this is the baseline every later phase attacks and defends.

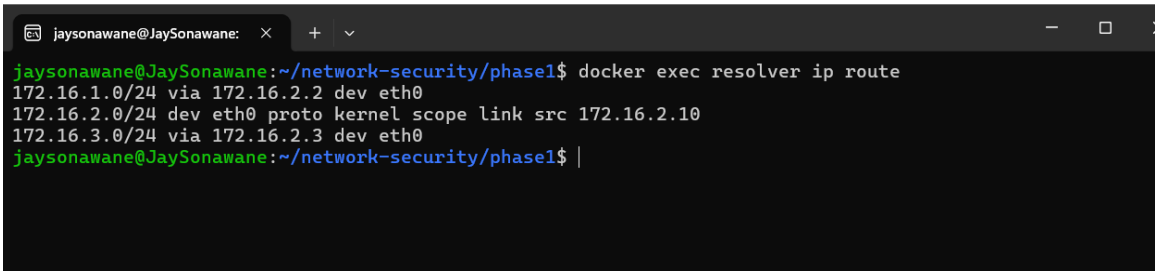
Design choices. All three Docker networks are declared `internal: true`, which means Docker gives them no gateway and no default route at all; the only way any container reaches another subnet is a static route its own `entrypoint.sh` installs, pointed at that segment's router. This makes the isolation property (the client cannot reach the authoritative subnet directly) true by construction rather than by an added firewall rule, and it is the same mechanism every later phase's diversion routes build on.

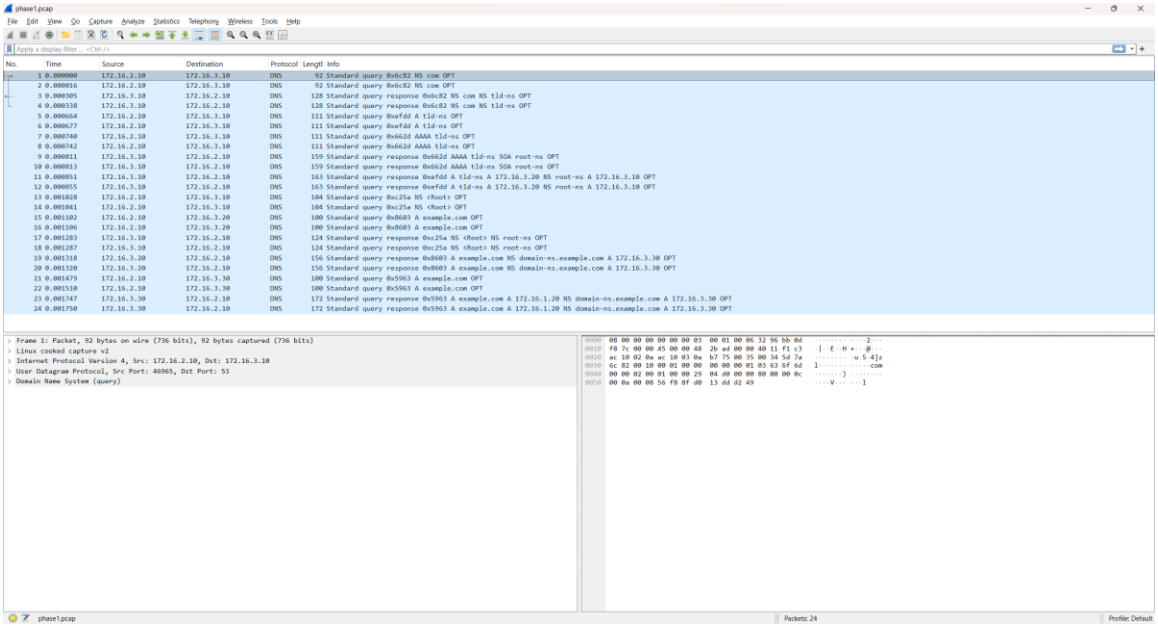
Verification evidence. `dig example.com` resolves to `172.16.1.20` and `curl` fetches the real page (`resolution.png`). A direct query from the client to `root-ns` times out, confirming the client has no route to the authoritative subnet at all (`isolation.png`). The static route installed on the resolver is shown in `resolver static route.png`. A full referral chain — root → `.com` → `example.com` — was captured on core-router and is shown in `referral-chain.png`, with an expanded view of one of the DNS packets in `expanded dns packet.png`. The overall topology is diagrammed in `phase1_topology.png`.

Screenshots.

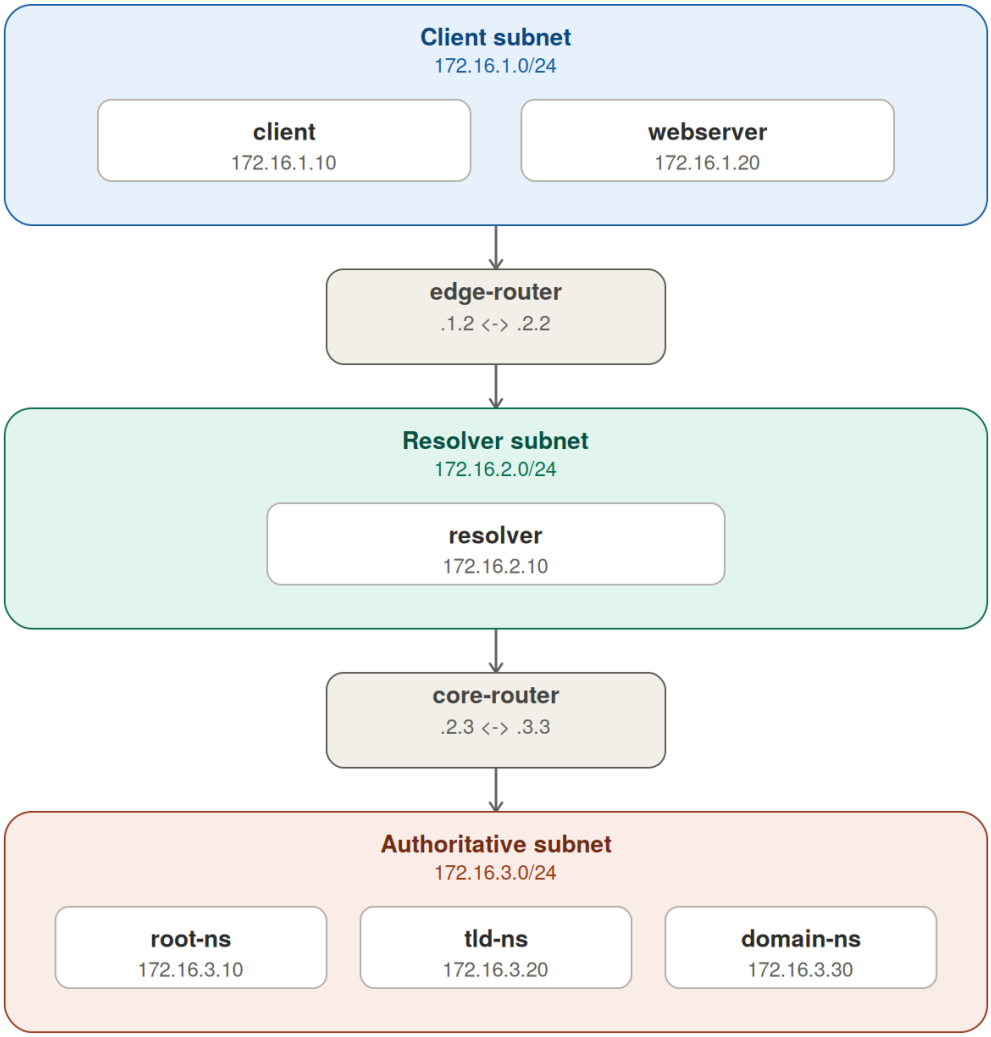





jaysonawane@JaySonawane:
jaysonawane@JaySonawane:~/network-security/phase1$ docker exec resolver ip route
172.16.1.0/24 via 172.16.2.2 dev eth0
172.16.2.0/24 dev eth0 proto kernel scope link src 172.16.2.10
172.16.3.0/24 via 172.16.2.3 dev eth0
jaysonawane@JaySonawane:~/network-security/phase1$


phase1.pcap
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
No. Time Source Destination Protocol Length Info
1 0.000000 172.16.2.10 172.16.3.10 DNS 92 Standard query 0x6c82 NS com OPT
2 0.000016 172.16.2.10 172.16.3.10 DNS 92 Standard query 0x6c82 NS com OPT
3 0.000305 172.16.3.10 172.16.2.10 DNS 128 Standard query response 0x6c82 NS com NS tld-ns OPT
4 0.000338 172.16.3.10 172.16.2.10 DNS 128 Standard query response 0x6c82 NS com NS tld-ns OPT
5 0.000664 172.16.2.10 172.16.3.10 DNS 111 Standard query 0xefdd A tld-ns OPT
6 0.000677 172.16.2.10 172.16.3.10 DNS 111 Standard query 0xefdd A tld-ns OPT
7 0.000740 172.16.2.10 172.16.3.10 DNS 111 Standard query 0x662d AAAA tld-ns OPT
8 0.000742 172.16.2.10 172.16.3.10 DNS 111 Standard query 0x662d AAAA tld-ns OPT
9 0.000811 172.16.3.10 172.16.2.10 DNS 159 Standard query response 0x662d AAAA tld-ns SOA root-ns OPT
10 0.000813 172.16.3.10 172.16.2.10 DNS 159 Standard query response 0x662d AAAA tld-ns SOA root-ns OPT
11 0.000851 172.16.3.10 172.16.2.10 DNS 163 Standard query response 0xefdd A tld-ns A 172.16.3.20 NS root-ns A 172.16.3.10 OPT
12 0.000855 172.16.3.10 172.16.2.10 DNS 163 Standard query response 0xefdd A tld-ns A 172.16.3.20 NS root-ns A 172.16.3.10 OPT
13 0.001028 172.16.2.10 172.16.3.10 DNS 104 Standard query 0xc25a NS <Root> OPT
14 0.001041 172.16.2.10 172.16.3.10 DNS 104 Standard query 0xc25a NS <Root> OPT
15 0.001102 172.16.2.10 172.16.3.20 DNS 100 Standard query 0x8603 A example.com OPT
16 0.001106 172.16.2.10 172.16.3.20 DNS 100 Standard query 0x8603 A example.com OPT
17 0.001283 172.16.3.10 172.16.2.10 DNS 124 Standard query response 0xc25a NS <Root> NS root-ns OPT
18 0.001287 172.16.3.10 172.16.2.10 DNS 124 Standard query response 0xc25a NS <Root> NS root-ns OPT
19 0.001318 172.16.3.20 172.16.2.10 DNS 156 Standard query response 0x8603 A example.com NS domain-ns.example.com A 172.16.3.30 OPT
20 0.001320 172.16.3.20 172.16.2.10 DNS 156 Standard query response 0x8603 A example.com NS domain-ns.example.com A 172.16.3.30 OPT
21 0.001479 172.16.2.10 172.16.3.30 DNS 100 Standard query 0x5963 A example.com OPT
22 0.001510 172.16.2.10 172.16.3.30 DNS 100 Standard query 0x5963 A example.com OPT
23 0.001747 172.16.3.30 172.16.2.10 DNS 172 Standard query response 0x5963 A example.com A 172.16.1.20 NS domain-ns.example.com A 172.16.3.30 OPT
24 0.001750 172.16.3.30 172.16.2.10 DNS 172 Standard query response 0x5963 A example.com A 172.16.1.20 NS domain-ns.example.com A 172.16.3.30 OPT
Frame 1: Packet, 92 bytes on wire (736 bits), 92 bytes captured (736 bits)
Linux cooked capture v2
Internet Protocol Version 4, Src: 172.16.2.10, Dst: 172.16.3.10
User Datagram Protocol, Src Port: 46965, Dst Port: 53
Domain Name System (query)
phase1.pcap
Packets: 24
Profile: Default


Wireshark · Packet 12 · phase1.pcap
Frame 12: Packet, 163 bytes on wire (1304 bits), 163 bytes captured (1304 bits)
Linux cooked capture v2
Internet Protocol Version 4, Src: 172.16.3.10, Dst: 172.16.2.10
User Datagram Protocol, Src Port: 53, Dst Port: 56118
Domain Name System (response)
No.: 12 · Time: 0.000855 · Source: 172.16.3.10 · Destination: 172.16.2.10 · Protocol: DNS · Length: 163 · Info: Standard query response 0xefdd A tld-ns A 172.16.3.20 NS root-ns A 172.16.3.10 OPT
Show packet bytes
Layout: Vertical (Stacked)
Close
Help



Report answers (§1.3).

- Resolved address, fetched page, and isolation check, with commands. `docker exec client dig example.com` returns `172.16.1.20`; `docker exec client curl http://example.com` returns the real page's HTML (`resolution.png`). `docker exec client dig @172.16.3.10 example.com +time=2 +tries=1` (a query sent directly to root-ns) times out — the authoritative subnet is not directly reachable from the client (`isolation.png`).
- From the capture, the Root → TLD → Domain referral: what is visible on the wire for each message? All three exchanges are plaintext UDP/53 and fully readable: the question name (`example.com`) and the DNS header's transaction ID are identical across all three queries the resolver sends (same question, reused ID pattern per outstanding query); root-ns's reply is a referral carrying no answer, only an AUTHORITY section naming the `.com` servers and an ADDITIONAL section with their glue addresses; tld-ns's reply is likewise a referral, naming domain-ns; only domain-ns's reply carries an ANSWER section with the actual A record for

`172.16.1.20` (`referral-chain.png`, with one message's full field breakdown in `expanded dns packet.png`).

- From the capture and `ip route`, how does the resolver's query reach root-ns? The resolver has a static route to 172.16.3.0/24 via core-router (172.16.2.3); packets to root-ns leave the resolver on that route, cross into the authoritative subnet through core-router (the only container with an interface in both `resolver_net` and `auth_net`), and arrive at root-ns on 172.16.3.10 (`resolver static route.png`).
- Given what the capture exposes, what could a node on the resolver's path do to this resolution? Since every query and answer is unauthenticated plaintext, any node sitting on the resolver's path to the authoritative servers can read the transaction ID, source port, and question of an outstanding query and craft a matching forged reply before (or instead of) the genuine one arrives — the resolver has no way to tell a forged answer from the real one. This is exactly the attack Phase 2 mounts.

Phase 2 — On-Path Response Forgery

Phase 1's topology is unchanged. Added: `rogue-web` (172.16.1.30, the defacement target), `attacker` (172.16.2.100, a libpcap on-path interceptor), and a single extra static route on core-router — `172.16.2.10/32 via 172.16.2.100` — diverting resolver-destined traffic through the attacker. No ARP spoofing anywhere.

Design choices. `attacker` has IP forwarding disabled at the kernel level, so nothing reaches the resolver unless the interceptor explicitly relays it — a deliberate "nothing crosses without the program's say" posture. It inspects every UDP/53 packet addressed to the resolver; only the final `A example.com` answer (exact name match, QTYPE A, an A record actually present) gets its 4 RDATA bytes overwritten to rogue-web's address when forging is toggled on. Every other packet — referrals, `www.example.com`, anything that fails the match — is relayed completely untouched. Both the IP and UDP checksums are recomputed from scratch on every relayed packet, forged or not; packets that transited a veth pair under real kernel forwarding can carry incomplete checksum-offload placeholders that look fine in a raw capture but get silently dropped by the receiving kernel, which was the actual cause of early SERVFAIL/timeout failures during development. Forging is toggled live via a file flag (`/tmp/forge.on`), no container restart needed.

Attack evidence. With forging off, `dig`/`curl` are fully transparent (`verify_transparent.png`). With forging on, `dig example.com` returns `172.16.1.30` and `curl` fetches the defaced page (`attack_success.png`), while `dig www.example.com` during the same attack window still returns the genuine address, untouched (`nontarget_unaffected.png`). The interceptor's own log of the forge is in `attacker_forge_log.png`, and the diversion route is shown in `diversion_route.png`. Wireshark captures contrast the genuine answer (`wireshark_genuine.png`) against the forged one (`wireshark_forged.png`).

Screenshots.

```
jay-sonawane@JayOmen16:~/network-security/phase2$ docker exec client dig example.com

; <<>> DiG 9.18.39-0ubuntu0.24.04.8-Ubuntu <<>> example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 39357
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: d88d57dea1d4ca15010000006ada824c21c2dfa4a1d3f9a8 (good)
;; QUESTION SECTION:
;example.com.                   IN      A

;; ANSWER SECTION:
example.com.            3600    IN      A       172.16.1.20

;; Query time: 5 msec
;; SERVER: 172.16.2.10#53(172.16.2.10) (UDP)
;; WHEN: Thu Oct 22 21:38:20 UTC 2026
;; MSG SIZE  rcvd: 84

jay-sonawane@JayOmen16:~/network-security/phase2$ docker exec client curl -s http://example.com
<!DOCTYPE html>
<html><head><title>example.com</title></head>
<body>
  <h1>This is the REAL example.com</h1>
  <p>Served by webserver (172.16.1.20).</p>
</body></html>
jay-sonawane@JayOmen16:~/network-security/phase2$
```

```
jay-sonawane@JayOmen16:~/network-security/phase2$ docker exec attacker touch /tmp/forge.on && docker compose restart resolver && sleep 3
[+] restart 0/1
 ⠸ Container resolver Restarting
jay-sonawane@JayOmen16:~/network-security/phase2$ docker exec client dig example.com

; <<>> DiG 9.18.39-0ubuntu0.24.04.8-Ubuntu <<>> example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 60512
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 052752091336fb35010000006ada8372a80ee73d339d15e5 (good)
;; QUESTION SECTION:
;example.com.                   IN      A

;; ANSWER SECTION:
example.com.            3600    IN      A       172.16.1.30

;; Query time: 6 msec
;; SERVER: 172.16.2.10#53(172.16.2.10) (UDP)
;; WHEN: Thu Oct 22 21:43:14 UTC 2026
;; MSG SIZE  rcvd: 84

jay-sonawane@JayOmen16:~/network-security/phase2$ docker exec client curl -s http://example.com
<!DOCTYPE html>
<html><head><title>example.com</title></head>
<body style="background:#2b0000;color:#fff;font-family:sans-serif;text-align:center;padding-top:80px;">
  <h1>DEFACED</h1>
  <p>This is rogue-web (172.16.1.30). example.com just got cache-poisoned.</p>
</body></html>
jay-sonawane@JayOmen16:~/network-security/phase2$
```

```
jay-sonawane@JayOmen16:~/network-security/phase2$ docker exec client dig www.example.com

; <<>> DiG 9.18.39-0ubuntu0.24.04.8-Ubuntu <<>> www.example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 1909
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 8da3510709f1b6dc010000006ada83ed79cbc1e7230cee94 (good)
;; QUESTION SECTION:
;www.example.com.               IN      A

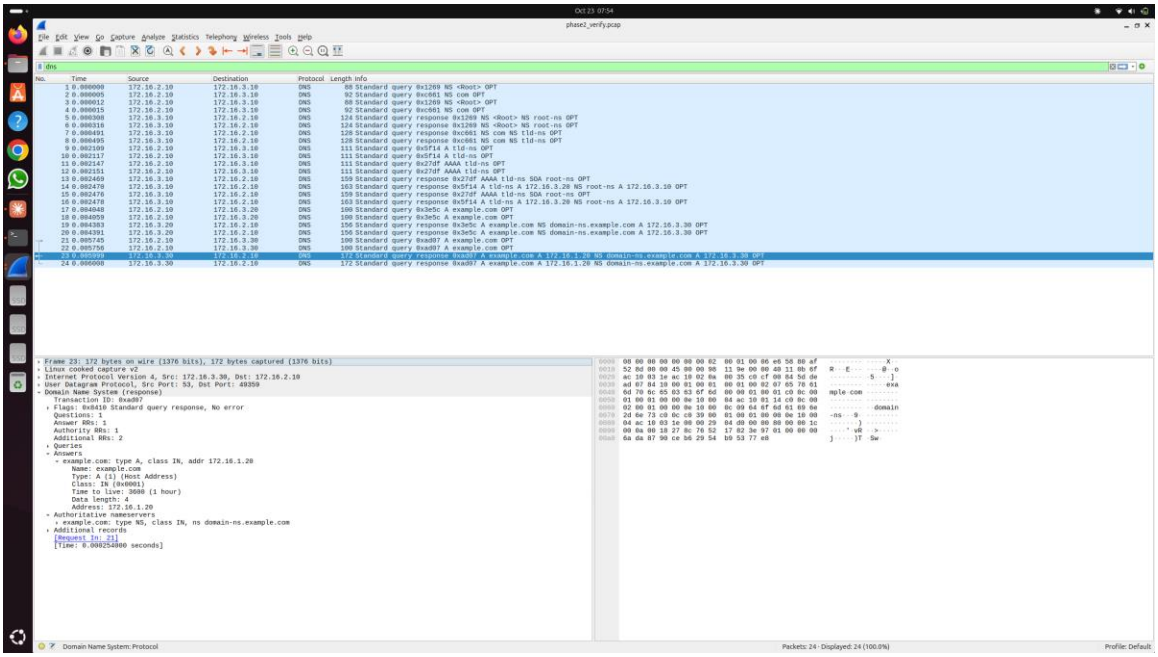
;; ANSWER SECTION:
www.example.com.        3600    IN      A       172.16.1.20

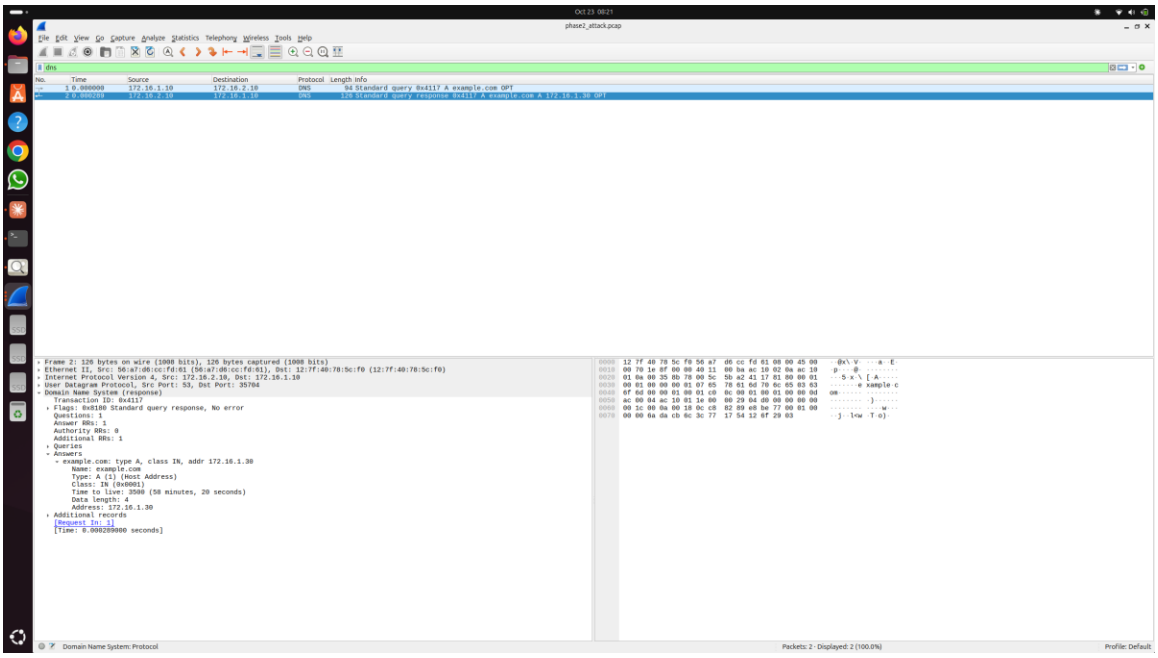
;; Query time: 0 msec
;; SERVER: 172.16.2.10#53(172.16.2.10) (UDP)
;; WHEN: Thu Oct 22 21:45:17 UTC 2026
;; MSG SIZE  rcvd: 88

jay-sonawane@JayOmen16:~/network-security/phase2$
```

```
jay-sonawane@JayOmen16:~/network-security/phase2$ docker compose logs attacker | grep forge
attacker  | [forge] example.com A -> rogue-web (txid 0x668c)
attacker  | [forge] example.com A -> rogue-web (txid 0x3cd3)
jay-sonawane@JayOmen16:~/network-security/phase2$
```

```
jay-sonawane@JayOmen16:~/network-security/phase2$ docker exec core-router ip route
172.16.2.0/24 dev eth1 proto kernel scope link src 172.16.2.3
172.16.2.10 via 172.16.2.100 dev eth1
172.16.3.0/24 dev eth0 proto kernel scope link src 172.16.3.3
jay-sonawane@JayOmen16:~/network-security/phase2$
```





Report answers (§2.4).

- Why does the attack succeed? Plain DNS has no field authenticating who actually produced a response — only that the reply matches the outstanding query (transaction ID, source port, question name/type). Being genuinely on-path, the attacker reads the real query and its transaction ID straight off the wire, so it never has to guess or race anything an off-path cache-poisoning attacker would.
- Why are non-target names left completely untouched? The forge condition is an exact match on the question name plus QTYPE A plus an A record actually present in the answer. `www.example.com` fails the name check and falls through to the identical passthrough path used for every other non-matching packet — there is no separate code path that could leak a partial modification into it.

Phase 3 — DNSSEC Chain of Trust

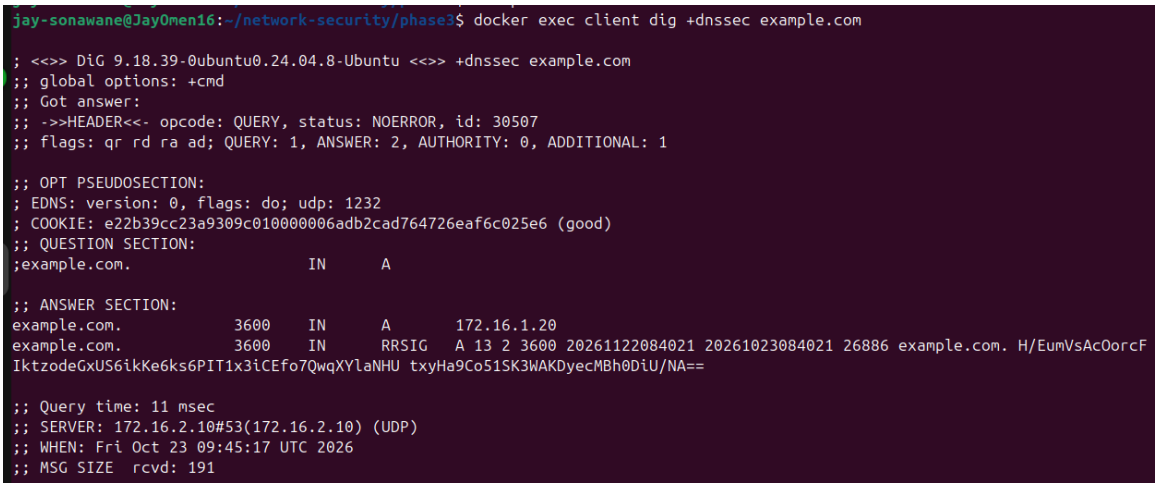
Identical topology to Phase 2, including the byte-for-byte unchanged `attacker` binary. The changes: root-ns, tld-ns, and domain-ns each sign their own zone (ECDSAP256SHA256), with the child's DS record embedded in the parent zone before the parent is signed — giving an unbroken chain root KSK → DS(com) → com KSK → DS(example.com) → example.com KSK. The resolver gets `dnssec-validation yes;` and a trust-anchors block holding the root zone's KSK, the one key trusted out of band; everything else is trusted transitively through the DS chain.

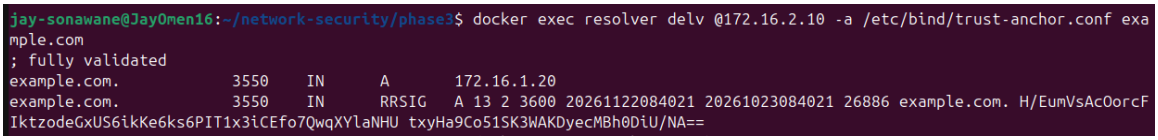
Design choices. Signing runs inside a disposable container via `scripts/sign-zones.sh`, always from the plain `.zone` source files, never from a previously-signed copy — so re-running it is always safe but also always generates brand-new keys, which means all four DNSSEC-related containers must be rebuilt together afterward (rebuilding only one leaves it trusting a different key than the others, breaking validation in a way indistinguishable from a real failure).

Verification evidence. `dig +dnssec example.com` shows the `ad` flag and a valid RRSIG (`verify_ad_flag.png`); `delv @172.16.2.10 -a trust-anchor.conf example.com` independently re-validates the whole chain from scratch and prints `; fully validated` (`verify_delv_validated.png`) — a stronger check than trusting the `ad` flag on a single `dig`.

Attack evidence. Re-running Phase 2's exact, unmodified attacker against this signed deployment now fails: `dig` returns SERVFAIL and `curl` gets nothing usable (`attack_servfail.png`, `attack_curl_empty.png`), captured at the resolver's own interface since that is the point that actually sees and rejects the forged packet (`attack_full_exchange.png`, `forged_response.png`, `resolver_servfail.png`).

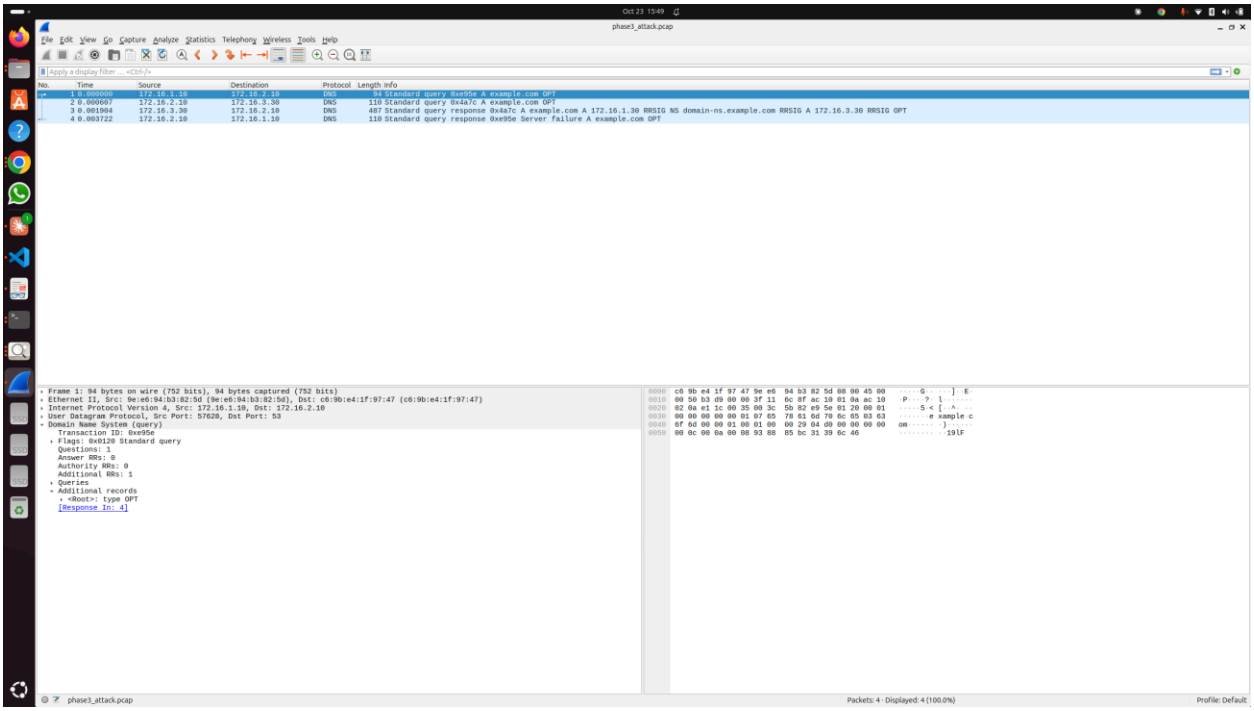
Screenshots.

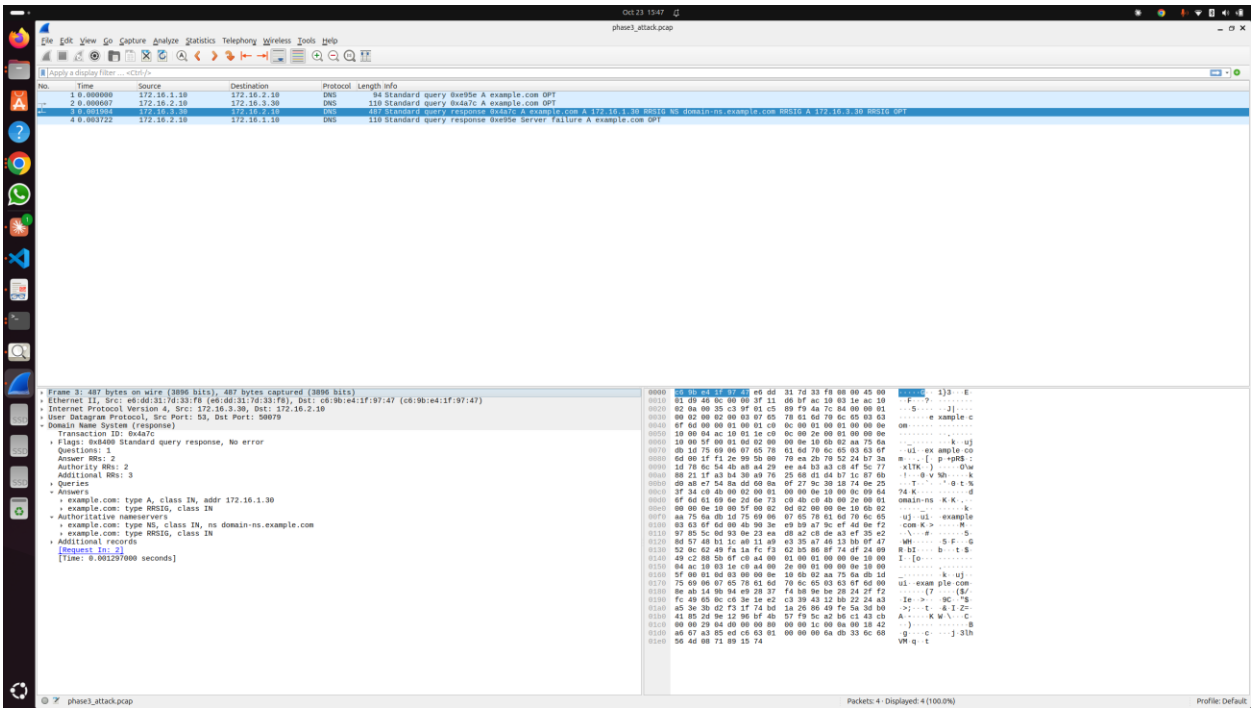


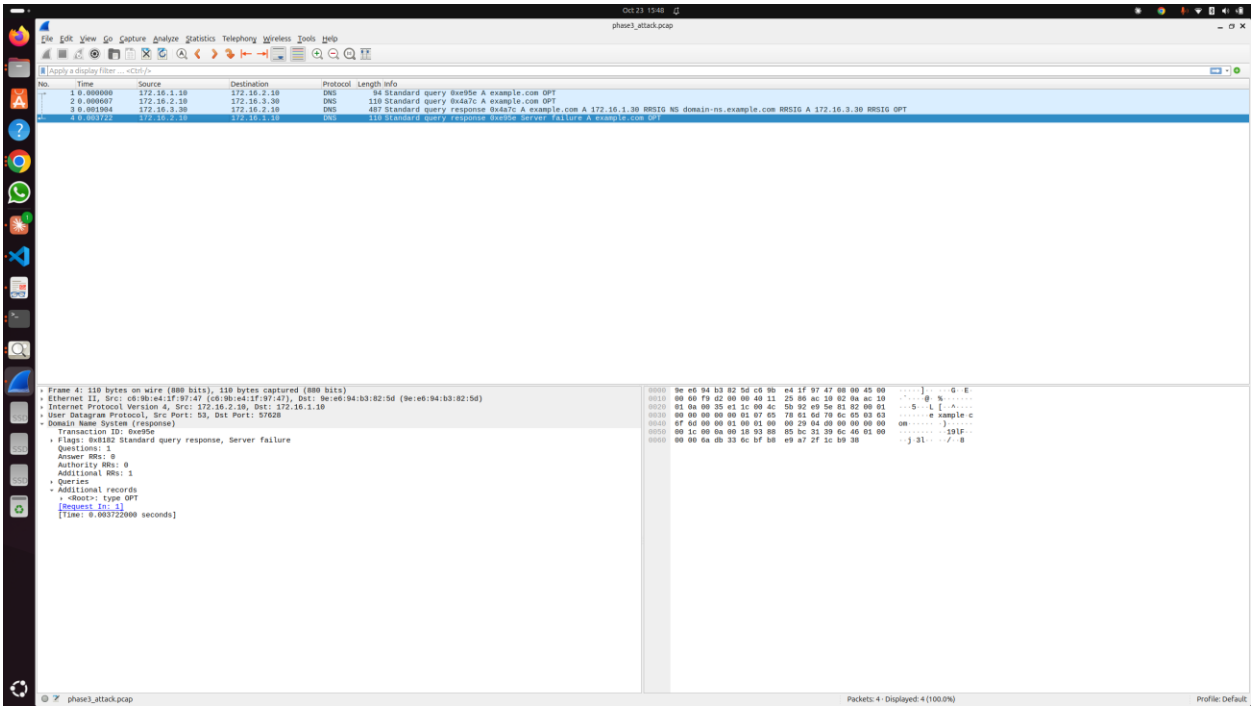




```
jay-sonawane@JayOmen16:~/network-security/phase3$ docker exec client curl -s http://example.com
jay-sonawane@JayOmen16:~/network-security/phase3$
```







Report answers (§3.4).

- What does the resolver do with the forged response, and why does this differ from Phase 2? The attacker still successfully intercepts and rewrites the A record — the interception itself is identical to Phase 2. What differs is that the forged record no longer carries a valid RRSIG matching the real domain-ns ZSK, since the attacker holds no private key anywhere in the chain. Validation fails specifically at the signature-check step, the forged answer is discarded, and the resolver returns SERVFAIL instead of caching a poisoned address.
- What is still visible on the wire? Every query and every response, forged or genuine, is still fully plaintext — DNSSEC adds no confidentiality whatsoever. It adds origin authentication and integrity for the data itself: it protects that the data was not tampered with, not that an on-path observer cannot read it.
- Why ECDSAP256SHA256, and what is the trade-off? Smaller keys and signatures than RSA at comparable security, which matters for response size and amplification risk. The cost is marginally more CPU work to verify per signature than RSA, and, outside this lab, somewhat less universal legacy-resolver support — a non-issue here since every resolver in the topology is one we control.
- Are private keys committed to the repository? No. `.gitignore` excludes `*.private` and `*.key` (both regenerate together from `sign-zones.sh`, and keeping stale ones around risks mismatching a freshly re-signed zone). Only the `.signed` zone files and `resolver/trust-anchor.conf` — which holds only the root KSK's public DNSKEY — are committed.

Phase 4 — Signature Stripping & Downgrade

Identical topology to Phase 3. The attacker gains a **strip mode**, usable independently of or alongside the existing forge mode. Strip mode matches any response whose question is `example.com` (any QTYPE), removes every RRSIG/DNSKEY/DS record wherever it appears in the message, and forges the A record's RDATA in the same pass. Unlike forge mode, this physically shortens the packet, so the DNS header's RR counts, UDP length, and IP total length are all recomputed.

Design choices — two deployment variants, same signing script. Run A (`sign-zones.sh`): the full chain, with example.com's DS genuinely published in com — strip mode must also strip that DS (and its RRSIG) from the referral to have any chance. Run B (`sign-zones.sh --omit-example-ds`): example.com is fully signed, but com simply never publishes its DS — the real-world operator mistake this phase is built around.

Attack evidence — Run A. With the DS genuinely published, stripping fails the same way Phase 3's attack did: SERVFAIL, no usable page (`runA_dig_servfail.png`, `runA_curl_empty.png`, `runA_servfail.png`, `runA_full_exchange.png`, `runA_stripped_answer.png`, `runA_attacker_log.png`). Deleting the RRSIG/DNSKEY/DS bytes does not make the zone look unsigned to the resolver — it already holds (or expects) a secure delegation, so an incomplete response for a zone it expects to be secure is treated as bogus, not as proof the zone is unsigned.

Attack evidence — Run B. With com never publishing example.com's DS, `dig +dnssec` first confirms the resolver now treats example.com as insecure (no `ad` flag, NOERROR) even before any attack. The same strip attack now succeeds: `dig` returns `172.16.1.30` and `curl` fetches the defaced page (`runB_dig_success.png`, `runB_curl_defaced.png`, `runB_client_answer.png`, `runB_full_exchange.png`, `runB_stripped_answer.png`, `runB_verify_insecure.png`, `runB_attacker_log.png`).

Screenshots.

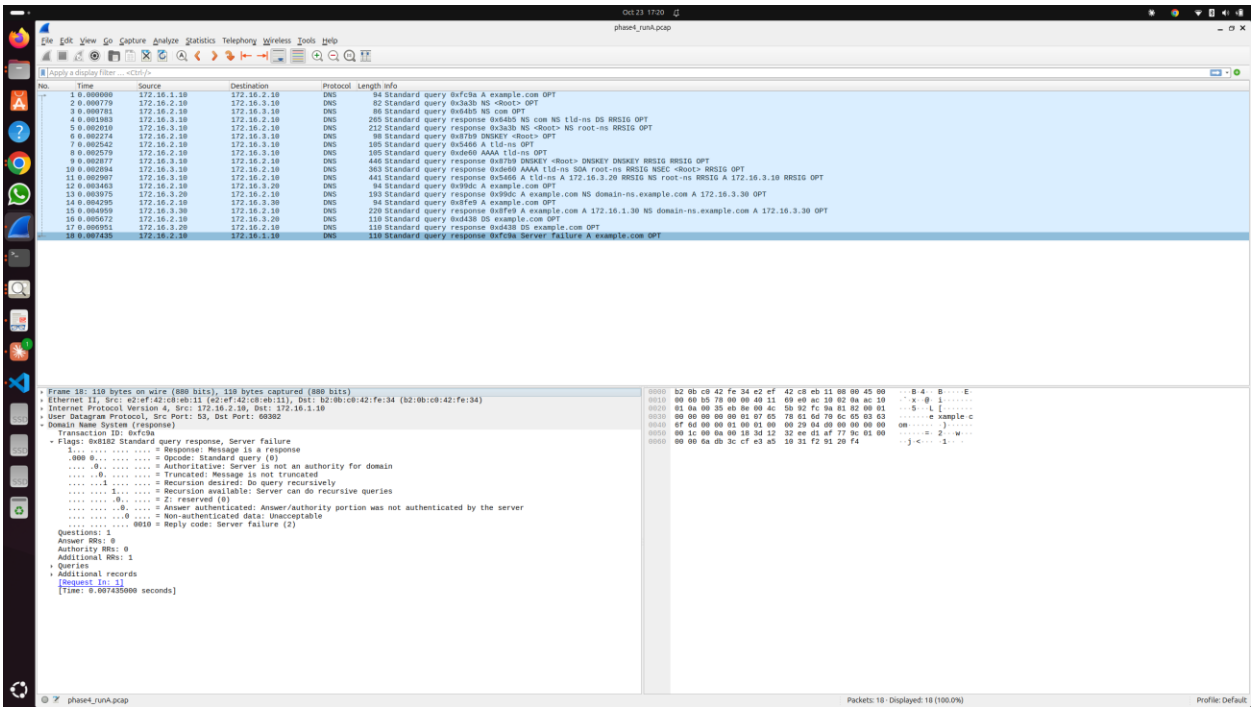
```
jay-sonawane@JayOmen16:~/network-security/phase4$ docker exec client dig example.com

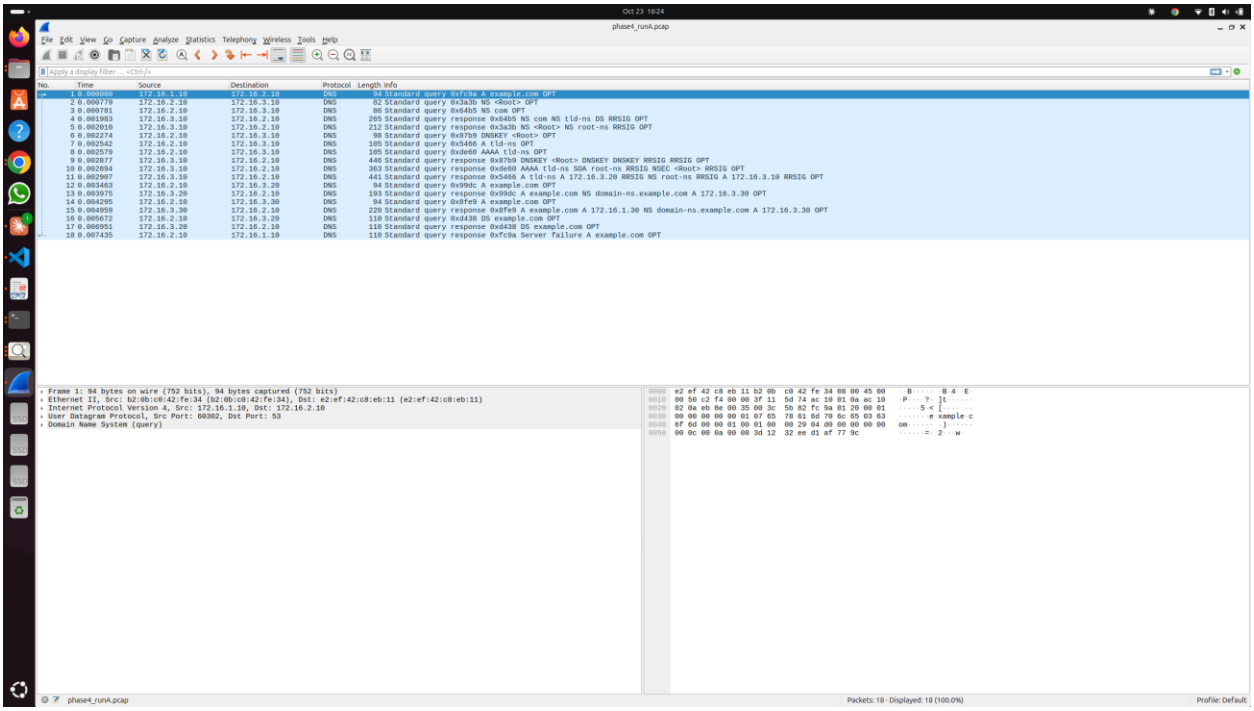
; <<>> DiG 9.18.39-0ubuntu0.24.04.8-Ubuntu <<>> example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: SERVFAIL, id: 25238
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 0, ADDITIONAL: 1

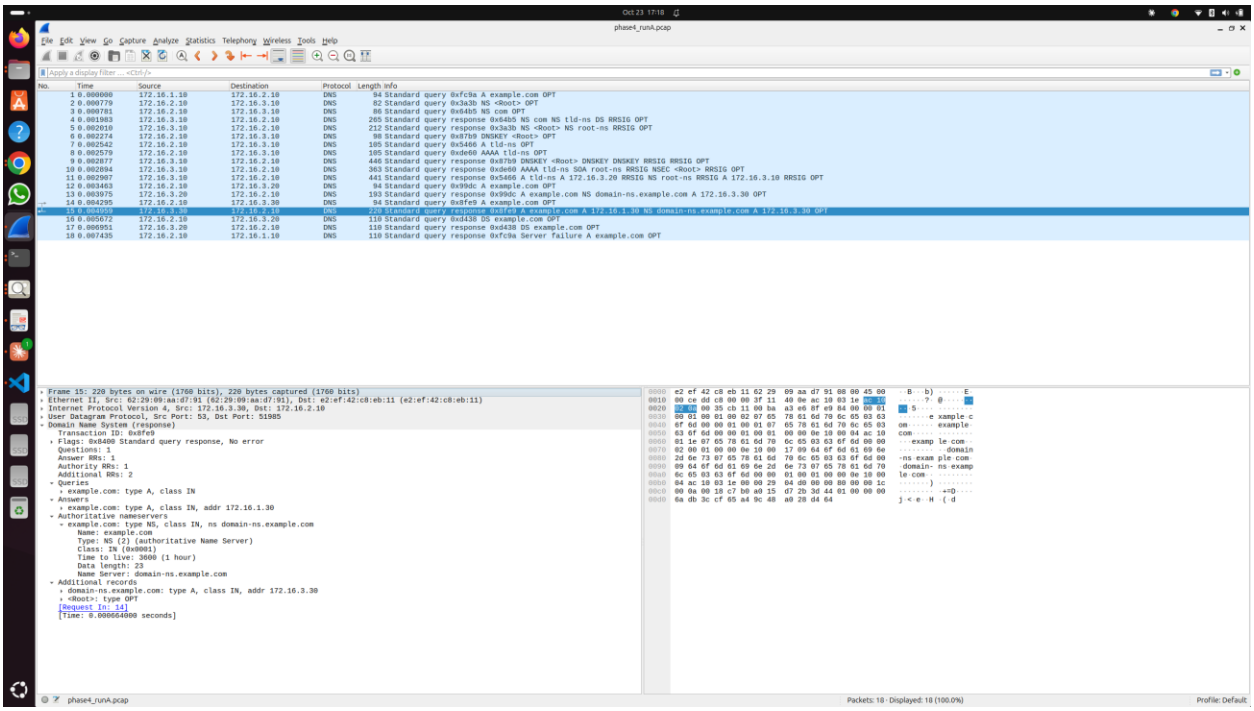
;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 3de2488297290f75010000006adb50f98eb6a343a6ffeb42 (good)
;; QUESTION SECTION:
;example.com.                   IN      A

;; Query time: 9 msec
;; SERVER: 172.16.2.10#53(172.16.2.10) (UDP)
;; WHEN: Fri Oct 23 12:20:09 UTC 2026
;; MSG SIZE  rcvd: 68
```

```
jay-sonawane@JayOmen16:~/network-security/phase4$ docker exec client curl -s http://example.com
jay-sonawane@JayOmen16:~/network-security/phase4$
```


Oct 23 17:20
phase4_runA.pcap
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
No. Time Source Destination Protocol Length Info
1 0.000000 172.16.1.10 172.16.2.10 DNS 94 Standard query 0xfc9a A example.com OPT
2 0.000779 172.16.2.10 172.16.3.10 DNS 82 Standard query 0x3a3b NS <Root> OPT
3 0.000781 172.16.2.10 172.16.3.10 DNS 86 Standard query 0x64b5 NS com OPT
4 0.001983 172.16.3.10 172.16.2.10 DNS 265 Standard query response 0x64b5 NS com NS tld-ns DS RRSIG OPT
5 0.002010 172.16.3.10 172.16.2.10 DNS 212 Standard query response 0x3a3b NS <Root> NS root-ns RRSIG OPT
6 0.002274 172.16.2.10 172.16.3.10 DNS 98 Standard query 0x87b9 DNSKEY <Root> OPT
7 0.002542 172.16.2.10 172.16.3.10 DNS 105 Standard query 0x5406 A tld-ns OPT
8 0.002579 172.16.2.10 172.16.3.10 DNS 105 Standard query 0xde60 AAAA tld-ns OPT
9 0.002877 172.16.3.10 172.16.2.10 DNS 446 Standard query response 0x87b9 DNSKEY <Root> DNSKEY DNSKEY RRSIG RRSIG OPT
10 0.002894 172.16.3.10 172.16.2.10 DNS 363 Standard query response 0xde60 AAAA tld-ns SOA root-ns RRSIG NSEC <Root> RRSIG OPT
11 0.002907 172.16.3.10 172.16.2.10 DNS 441 Standard query response 0x5406 A tld-ns A 172.16.3.20 RRSIG NS root-ns RRSIG A 172.16.3.10 RRSIG OPT
12 0.003463 172.16.2.10 172.16.3.20 DNS 94 Standard query 0x99dc A example.com OPT
13 0.003975 172.16.3.20 172.16.2.10 DNS 193 Standard query response 0x99dc A example.com NS domain-ns.example.com A 172.16.3.30 OPT
14 0.004295 172.16.2.10 172.16.3.30 DNS 94 Standard query 0x8fe9 A example.com OPT
15 0.004959 172.16.3.30 172.16.2.10 DNS 220 Standard query response 0x8fe9 A example.com A 172.16.1.30 NS domain-ns.example.com A 172.16.3.30 OPT
16 0.005672 172.16.2.10 172.16.3.20 DNS 110 Standard query 0xd438 DS example.com OPT
17 0.006951 172.16.3.20 172.16.2.10 DNS 110 Standard query response 0xd438 DS example.com OPT
18 0.007435 172.16.2.10 172.16.1.10 DNS 110 Standard query response 0xfc9a Server failure A example.com OPT
Frame 18: 110 bytes on wire (880 bits), 110 bytes captured (880 bits)
Ethernet II, Src: e2:ef:42:c8:eb:11 (e2:ef:42:c8:eb:11), Dst: b2:0b:c0:42:fe:34 (b2:0b:c0:42:fe:34)
Internet Protocol Version 4, Src: 172.16.2.10, Dst: 172.16.1.10
User Datagram Protocol, Src Port: 53, Dst Port: 60302
Domain Name System (response)
Transaction ID: 0xfc9a
Flags: 0x8182 Standard query response, Server failure
1... .... = Response: Message is a response
.000 0... .... = Opcode: Standard query (0)
.... .0.. .... = Authoritative: Server is not an authority for domain
.... ..0. .... = Truncated: Message is not truncated
....1 .... = Recursion desired: Do query recursively
.... 1... .... = Recursion available: Server can do recursive queries
.... .0.. .... = Z: reserved (0)
....0. .... = Answer authenticated: Answer/authority portion was not authenticated by the server
....0 .... = Non-authenticated data: Unacceptable
.... 0010 = Reply code: Server failure (2)
Questions: 1
Answer RRs: 0
Authority RRs: 0
Additional RRs: 1
Queries
Additional records
[Request In: 11]
[Time: 0.007435000 seconds]
0000 b2 0b c0 42 fe 34 e2 ef 42 c8 eb 11 08 00 45 00
0010 00 60 b5 78 00 00 40 11 69 e0 ac 10 02 0a ac 10
0020 01 0a 00 35 eb 8e 00 4c 5b 92 fc 9a 81 82 00 01
0030 00 00 00 00 00 01 07 65 78 61 6d 70 6c 65 03 63
0040 6f 6d 00 00 01 00 01 00 00 29 04 d0 00 00 00 00
0050 00 1c 00 0a 00 18 3d 12 32 ee d1 af 77 9c 01 00
0060 00 00 6a db 3c cf e3 a5 10 31 f2 91 20 f4
phase4_runA.pcap
Packets: 18 · Displayed: 18 (100.0%)
Profile: Default


Oct 23 16:24
phase4_runA.pcap
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
No. Time Source Destination Protocol Length Info
1 0.000000 172.16.1.10 172.16.2.10 DNS 94 Standard query 0xfc9a A example.com OPT
2 0.000779 172.16.2.10 172.16.3.10 DNS 82 Standard query 0x3a3b NS <Root> OPT
3 0.000781 172.16.2.10 172.16.3.10 DNS 86 Standard query 0x64b5 NS com OPT
4 0.001983 172.16.3.10 172.16.2.10 DNS 265 Standard query response 0x64b5 NS com NS tld-ns DS RRSIG OPT
5 0.002010 172.16.3.10 172.16.2.10 DNS 212 Standard query response 0x3a3b NS <Root> NS root-ns RRSIG OPT
6 0.002274 172.16.2.10 172.16.3.10 DNS 98 Standard query 0x87b9 DNSKEY <Root> OPT
7 0.002542 172.16.2.10 172.16.3.10 DNS 105 Standard query 0x5406 A tld-ns OPT
8 0.002579 172.16.2.10 172.16.3.10 DNS 105 Standard query 0xde60 AAAA tld-ns OPT
9 0.002877 172.16.3.10 172.16.2.10 DNS 446 Standard query response 0x87b9 DNSKEY <Root> DNSKEY DNSKEY RRSIG RRSIG OPT
10 0.002894 172.16.3.10 172.16.2.10 DNS 363 Standard query response 0xde60 AAAA tld-ns SOA root-ns RRSIG NSEC <Root> RRSIG OPT
11 0.002907 172.16.3.10 172.16.2.10 DNS 441 Standard query response 0x5406 A tld-ns A 172.16.3.20 RRSIG NS root-ns RRSIG A 172.16.3.10 RRSIG OPT
12 0.003463 172.16.2.10 172.16.3.20 DNS 94 Standard query 0x99dc A example.com OPT
13 0.003975 172.16.3.20 172.16.2.10 DNS 193 Standard query response 0x99dc A example.com NS domain-ns.example.com A 172.16.3.30 OPT
14 0.004295 172.16.2.10 172.16.3.30 DNS 94 Standard query 0x8fe9 A example.com OPT
15 0.004959 172.16.3.30 172.16.2.10 DNS 220 Standard query response 0x8fe9 A example.com A 172.16.1.30 NS domain-ns.example.com A 172.16.3.30 OPT
16 0.005672 172.16.2.10 172.16.3.20 DNS 110 Standard query 0xd438 DS example.com OPT
17 0.006951 172.16.3.20 172.16.2.10 DNS 110 Standard query response 0xd438 DS example.com OPT
18 0.007435 172.16.2.10 172.16.1.10 DNS 110 Standard query response 0xfc9a Server failure A example.com OPT
Frame 1: 94 bytes on wire (752 bits), 94 bytes captured (752 bits)
Ethernet II, Src: b2:0b:c0:42:fe:34 (b2:0b:c0:42:fe:34), Dst: e2:ef:42:c8:eb:11 (e2:ef:42:c8:eb:11)
Internet Protocol Version 4, Src: 172.16.1.10, Dst: 172.16.2.10
User Datagram Protocol, Src Port: 60302, Dst Port: 53
Domain Name System (query)
0000 e2 ef 42 c8 eb 11 b2 0b c0 42 fe 34 08 00 45 00
0010 00 50 c2 f4 00 00 3f 11 5d 74 ac 10 01 0a ac 10
0020 02 0a eb 8e 00 35 00 3c 5b 82 fc 9a 01 20 00 01
0030 00 00 00 00 00 01 07 65 78 61 6d 70 6c 65 03 63
0040 6f 6d 00 00 01 00 01 00 00 29 04 d0 00 00 00 00
0050 00 0c 00 0a 00 08 3d 12 32 ee d1 af 77 9c
phase4_runA.pcap
Packets: 18 · Displayed: 18 (100.0%)
Profile: Default




```
jay-sonawane@JayOmen16:~/network-security/phase4$ docker logs attacker --tail 30
[strip] removed type=46 owner=example.com (txid 0x3943)
[strip] removed type=46 owner=example.com (txid 0x3943)
[attacker] resolver MAC changed -> 5e:f9:55:61:31:a7
[attacker] resolver MAC changed -> da:e3:d0:ab:70:20
[attacker] resolver MAC changed -> 42:96:46:51:77:bc
[strip+forge] example.com A -> rogue-web (txid 0x0251)
[strip] removed type=46 owner=example.com (txid 0x0251)
[strip] removed type=46 owner=example.com (txid 0x0251)
[strip] removed type=48 owner=example.com (txid 0x93f3)
[strip] removed type=48 owner=example.com (txid 0x93f3)
[strip] removed type=48 owner=example.com (txid 0xce7c)
[strip] removed type=48 owner=example.com (txid 0xce7c)
[attacker] resolver MAC changed -> b2:9b:53:17:03:fb
[attacker] resolver MAC changed -> da:af:9c:89:92:90
[strip] removed type=43 owner=example.com (txid 0xb8a5)
[strip] removed type=46 owner=example.com (txid 0xb8a5)
[strip+forge] example.com A -> rogue-web (txid 0x0dab)
[strip] removed type=46 owner=example.com (txid 0x0dab)
[strip] removed type=46 owner=example.com (txid 0x0dab)
[strip] removed type=43 owner=example.com (txid 0x39c0)
[strip] removed type=46 owner=example.com (txid 0x39c0)
[strip+forge] example.com A -> rogue-web (txid 0x0b63)
[strip] removed type=46 owner=example.com (txid 0x0b63)
[strip] removed type=46 owner=example.com (txid 0x0b63)
[strip] removed type=48 owner=example.com (txid 0xc884)
[strip] removed type=48 owner=example.com (txid 0xc884)
[strip] removed type=43 owner=example.com (txid 0xf2fb)
[strip] removed type=46 owner=example.com (txid 0xf2fb)
[strip] removed type=48 owner=example.com (txid 0x7144)
[strip] removed type=48 owner=example.com (txid 0x7144)
jay-sonawane@JayOmen16:~/network-security/phase4$
```

```
jay-sonawane@JayOmen16:~/network-security/phase4$ docker exec client dig +dnssec example.com

; <<>> DiG 9.18.39-0ubuntu0.24.04.8-Ubuntu <<>> +dnssec example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 41448
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags: do; udp: 1232
; COOKIE: 3c5bb07b1120d8a7010000006adb4e29d06313d434f37b75 (good)
;; QUESTION SECTION:
;example.com.                   IN      A

;; ANSWER SECTION:
example.com.            3600    IN      A       172.16.1.20
example.com.            3600    IN      RRSIG   A 13 2 3600 20261122105253 20261023105253 30885 example.com. x0ZGFjjO/AHsMu
ajKaRRHFoc8XQNdPAv5d58Unrbchw6AV9B5kQPJWCF K0aP3EZ7/D8LC55fpDpR3CtfEa5MpQ==

;; Query time: 10 msec
;; SERVER: 172.16.2.10#53(172.16.2.10) (UDP)
;; WHEN: Fri Oct 23 12:08:09 UTC 2026
;; MSG SIZE  rcvd: 191

jay-sonawane@JayOmen16:~/network-security/phase4$
```

```
jay-sonawane@JayOmen16:~/network-security/phase4$ docker exec client dig example.com

; <<>> DiG 9.18.39-0ubuntu0.24.04.8-Ubuntu <<>> example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 3223
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

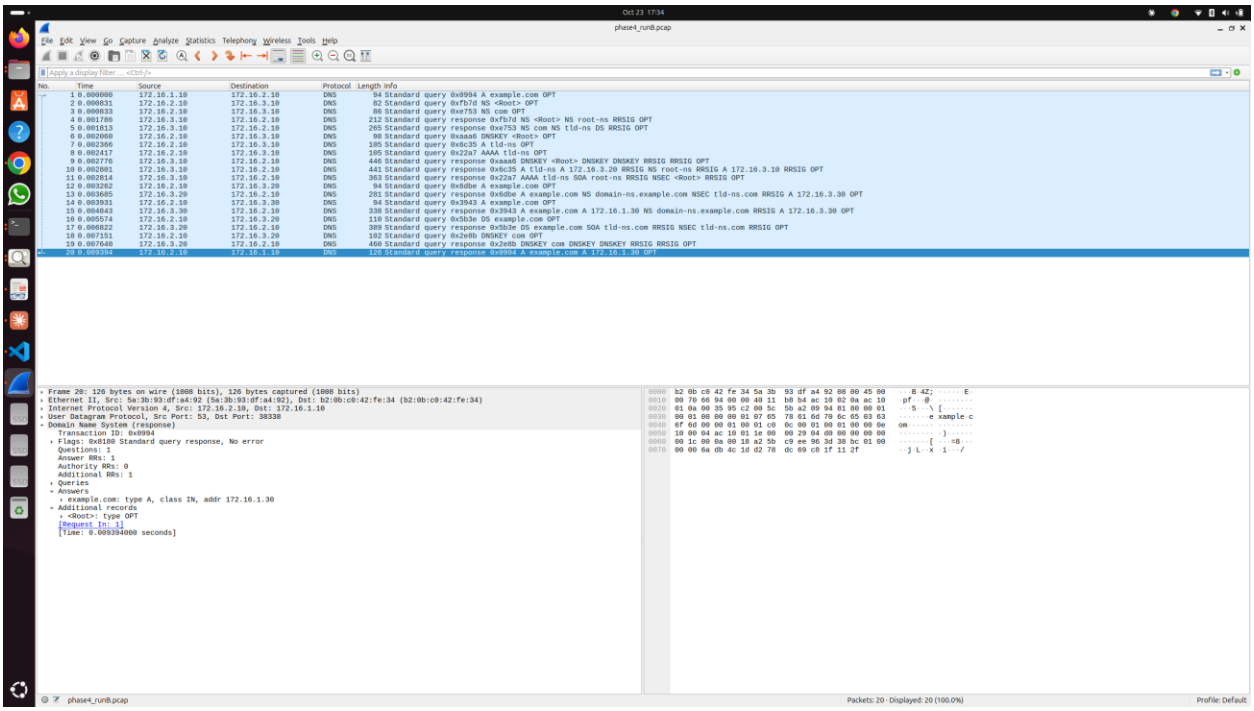
;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 2f2820148a6996b2010000006adb4fa58ce681216c38a2fb (good)
;; QUESTION SECTION:
;example.com.                   IN      A

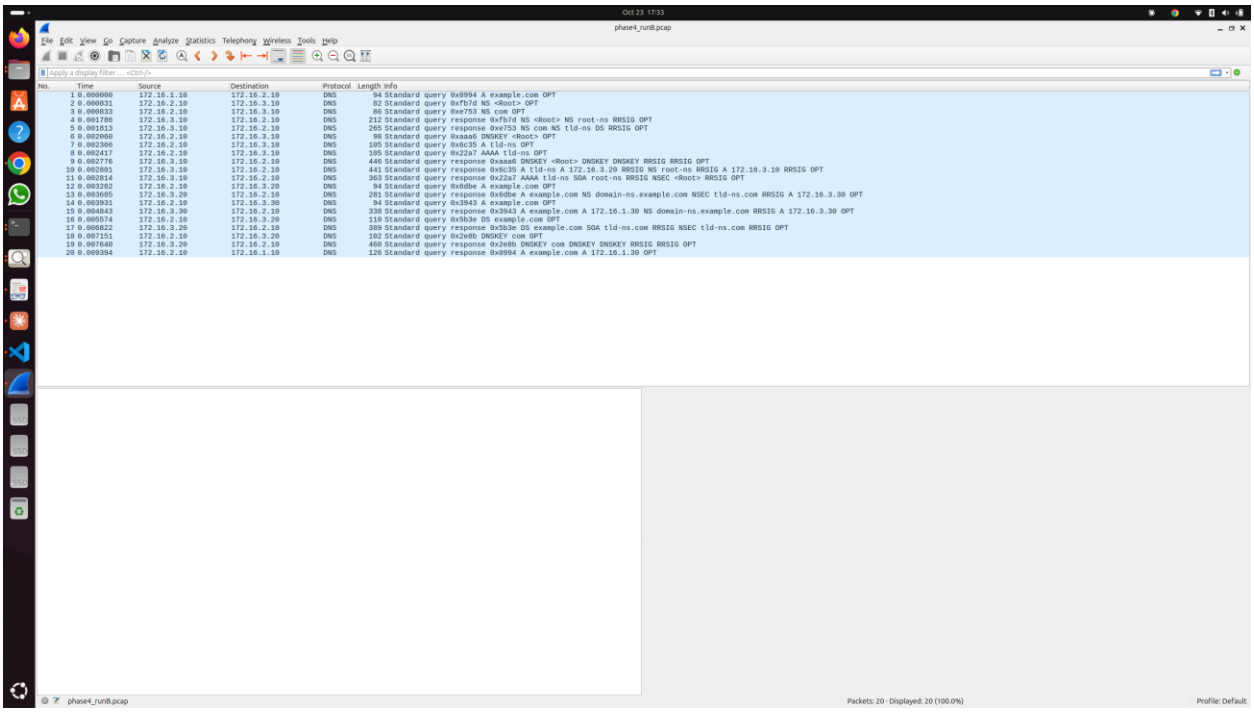
;; ANSWER SECTION:
example.com.            3600    IN      A       172.16.1.30

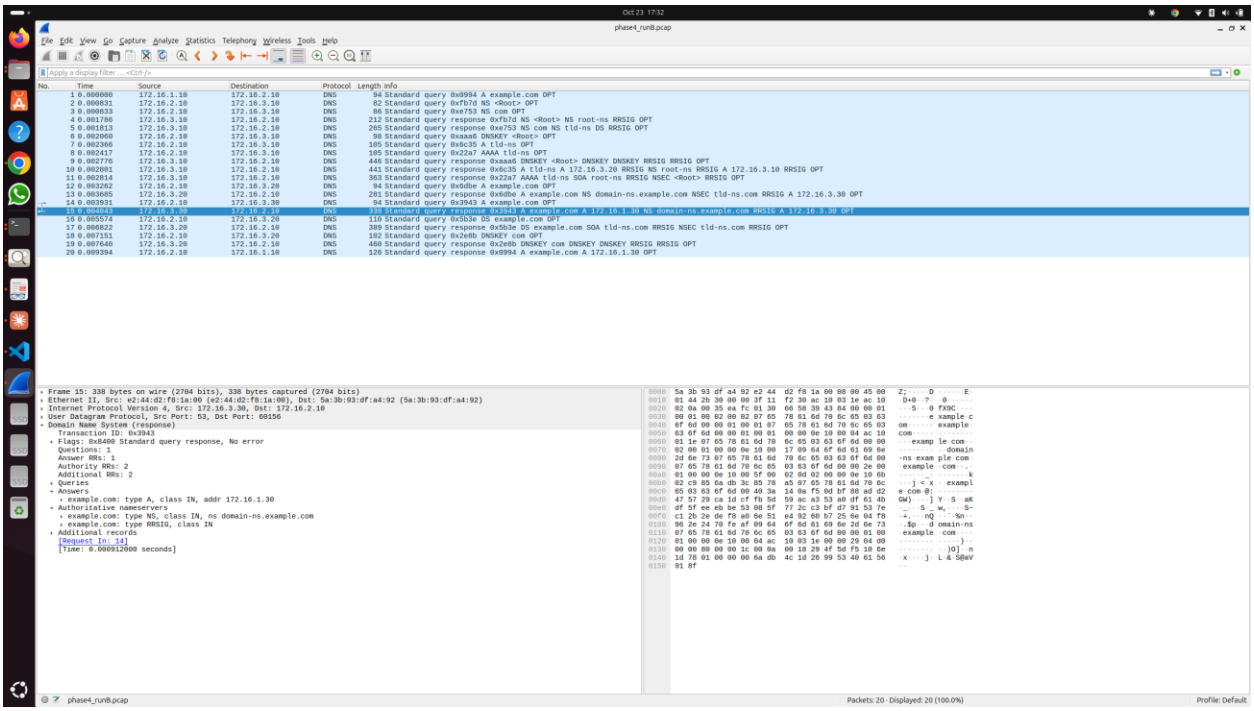
;; Query time: 11 msec
;; SERVER: 172.16.2.10#53(172.16.2.10) (UDP)
;; WHEN: Fri Oct 23 12:14:29 UTC 2026
;; MSG SIZE  rcvd: 84

jay-sonawane@JayOmen16:~/network-security/phase4$
```

```
jay-sonawane@JayOmen16:~/network-security/phase4$ docker exec client curl -s http://example.com
<!DOCTYPE html>
<html><head><title>example.com</title></head>
<body style="background:#2b0000;color:#fff;font-family:sans-serif;text-align:center;padding-top:80px;">
  <h1>DEFACED</h1>
  <p>This is rogue-web (172.16.1.30). example.com just got cache-poisoned.</p>
</body></html>
jay-sonawane@JayOmen16:~/network-security/phase4$
```


Oct 23 17:34
phase4_runB.pcap
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
No. Time Source Destination Protocol Length Info
1 0.000000 172.16.1.10 172.16.2.10 DNS 94 Standard query 0x0994 A example.com OPT
2 0.000831 172.16.2.10 172.16.3.10 DNS 82 Standard query 0xfb7d NS <Root> OPT
3 0.000833 172.16.2.10 172.16.3.10 DNS 86 Standard query 0xe753 NS com OPT
4 0.001786 172.16.3.10 172.16.2.10 DNS 212 Standard query response 0xfb7d NS <Root> NS root-ns RRSIG OPT
5 0.001813 172.16.3.10 172.16.2.10 DNS 265 Standard query response 0xe753 NS com NS tld-ns DS RRSIG OPT
6 0.002060 172.16.2.10 172.16.3.10 DNS 98 Standard query 0xaaa6 DNSKEY <Root> OPT
7 0.002366 172.16.2.10 172.16.3.10 DNS 105 Standard query 0x6c35 A tld-ns OPT
8 0.002417 172.16.2.10 172.16.3.10 DNS 105 Standard query 0x22a7 AAAA tld-ns OPT
9 0.002776 172.16.3.10 172.16.2.10 DNS 446 Standard query response 0xaaa6 DNSKEY <Root> DNSKEY DNSKEY RRSIG RRSIG OPT
10 0.002801 172.16.3.10 172.16.2.10 DNS 441 Standard query response 0x6c35 A tld-ns A 172.16.3.20 RRSIG NS root-ns RRSIG A 172.16.3.10 RRSIG OPT
11 0.002814 172.16.3.10 172.16.2.10 DNS 363 Standard query response 0x22a7 AAAA tld-ns SOA root-ns RRSIG NSEC <Root> RRSIG OPT
12 0.003262 172.16.2.10 172.16.3.20 DNS 94 Standard query 0x6dbe A example.com OPT
13 0.003685 172.16.3.20 172.16.2.10 DNS 281 Standard query response 0x6dbe A example.com NS domain-ns.example.com NSEC tld-ns.com RRSIG A 172.16.3.30 OPT
14 0.003931 172.16.2.10 172.16.3.30 DNS 94 Standard query 0x3943 A example.com OPT
15 0.004843 172.16.3.30 172.16.2.10 DNS 338 Standard query response 0x3943 A example.com A 172.16.1.30 NS domain-ns.example.com RRSIG A 172.16.3.30 OPT
16 0.005574 172.16.2.10 172.16.3.20 DNS 110 Standard query 0x5b3e DS example.com OPT
17 0.006822 172.16.3.20 172.16.2.10 DNS 389 Standard query response 0x5b3e DS example.com SOA tld-ns.com RRSIG NSEC tld-ns.com RRSIG OPT
18 0.007151 172.16.2.10 172.16.3.20 DNS 102 Standard query 0x2e8b DNSKEY com OPT
19 0.007640 172.16.3.20 172.16.2.10 DNS 460 Standard query response 0x2e8b DNSKEY com DNSKEY DNSKEY RRSIG RRSIG OPT
20 0.009394 172.16.2.10 172.16.1.10 DNS 126 Standard query response 0x0994 A example.com A 172.16.1.30 OPT
Frame 20: 126 bytes on wire (1008 bits), 126 bytes captured (1008 bits)
Ethernet II, Src: 5a:3b:93:df:a4:92 (5a:3b:93:df:a4:92), Dst: b2:0b:c0:42:fe:34 (b2:0b:c0:42:fe:34)
Internet Protocol Version 4, Src: 172.16.2.10, Dst: 172.16.1.10
User Datagram Protocol, Src Port: 53, Dst Port: 38338
Domain Name System (response)
Transaction ID: 0x0994
Flags: 0x8180 Standard query response, No error
Questions: 1
Answer RRs: 1
Authority RRs: 0
Additional RRs: 1
Queries
Answers
example.com: type A, class IN, addr 172.16.1.30
Additional records
<Root>: type OPT
[Request In: 1]
[Time: 0.009394000 seconds]
b2 0b c0 42 fe 34 5a 3b 93 df a4 92 08 00 45 00
00 70 66 94 00 00 40 11 b8 b4 ac 10 02 0a ac 10
01 0a 00 35 95 c2 00 5c 5b a2 09 94 81 80 00 01
00 01 00 00 00 01 07 65 78 61 6d 70 6c 65 03 63
6f 6d 00 00 01 00 01 c0 0c 00 01 00 01 00 00 0e
10 00 04 ac 10 01 1e 00 00 29 04 d0 00 00 00 00
00 1c 00 0a 00 18 a2 5b c9 ee 96 3d 38 bc 01 00
00 00 6a db 4c 1d d2 78 dc 69 c8 1f 11 2f
phase4_runB.pcap
Packets: 20 · Displayed: 20 (100.0%)
Profile: Default


Oct 23 17:33
phase4_runB.pcap
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
No. Time Source Destination Protocol Length Info
1 0.000000 172.16.1.10 172.16.2.10 DNS 94 Standard query 0x0994 A example.com OPT
2 0.000831 172.16.2.10 172.16.3.10 DNS 82 Standard query 0xfb7d NS <Root> OPT
3 0.000833 172.16.2.10 172.16.3.10 DNS 86 Standard query 0xe753 NS com OPT
4 0.001786 172.16.3.10 172.16.2.10 DNS 212 Standard query response 0xfb7d NS <Root> NS root-ns RRSIG OPT
5 0.001813 172.16.3.10 172.16.2.10 DNS 265 Standard query response 0xe753 NS com NS tld-ns DS RRSIG OPT
6 0.002060 172.16.2.10 172.16.3.10 DNS 98 Standard query 0xaaa6 DNSKEY <Root> OPT
7 0.002366 172.16.2.10 172.16.3.10 DNS 105 Standard query 0x6c35 A tld-ns OPT
8 0.002417 172.16.2.10 172.16.3.10 DNS 105 Standard query 0x22a7 AAAA tld-ns OPT
9 0.002776 172.16.3.10 172.16.2.10 DNS 446 Standard query response 0xaaa6 DNSKEY <Root> DNSKEY DNSKEY RRSIG RRSIG OPT
10 0.002801 172.16.3.10 172.16.2.10 DNS 441 Standard query response 0x6c35 A tld-ns A 172.16.3.20 RRSIG NS root-ns RRSIG A 172.16.3.10 RRSIG OPT
11 0.002814 172.16.3.10 172.16.2.10 DNS 363 Standard query response 0x22a7 AAAA tld-ns SOA root-ns RRSIG NSEC <Root> RRSIG OPT
12 0.003262 172.16.2.10 172.16.3.20 DNS 94 Standard query 0x6dbe A example.com OPT
13 0.003685 172.16.3.20 172.16.2.10 DNS 281 Standard query response 0x6dbe A example.com NS domain-ns.example.com NSEC tld-ns.com RRSIG A 172.16.3.30 OPT
14 0.003931 172.16.2.10 172.16.3.30 DNS 94 Standard query 0x3943 A example.com OPT
15 0.004843 172.16.3.30 172.16.2.10 DNS 338 Standard query response 0x3943 A example.com A 172.16.1.30 NS domain-ns.example.com RRSIG A 172.16.3.30 OPT
16 0.005574 172.16.2.10 172.16.3.20 DNS 110 Standard query 0x5b3e DS example.com OPT
17 0.006822 172.16.3.20 172.16.2.10 DNS 389 Standard query response 0x5b3e DS example.com SOA tld-ns.com RRSIG NSEC tld-ns.com RRSIG OPT
18 0.007151 172.16.2.10 172.16.3.20 DNS 102 Standard query 0x2e8b DNSKEY com OPT
19 0.007640 172.16.3.20 172.16.2.10 DNS 460 Standard query response 0x2e8b DNSKEY com DNSKEY DNSKEY RRSIG RRSIG OPT
20 0.009394 172.16.2.10 172.16.1.10 DNS 126 Standard query response 0x0994 A example.com A 172.16.1.30 OPT
phase4_runB.pcap
Packets: 20 · Displayed: 20 (100.0%)
Profile: Default




```
jay-sonawane@JayOmen16:~/network-security/phase4$ docker logs attacker --tail 30
net.ipv4.ip_forward = 0
[attacker] priming ARP for the real resolver (172.16.2.10)...
[attacker] resolver MAC = 32:39:85:28:1f:dc
[attacker] up on eth0, target=example.com, rogue=172.16.1.30, resolver=172.16.2.10, forging=OFF, stripping=OFF
[attacker] resolver MAC changed -> 62:dc:e8:9b:bd:0d
[strip+forge] example.com A -> rogue-web (txid 0x310d)
[strip] removed type=46 owner=example.com (txid 0x310d)
[strip] removed type=46 owner=example.com (txid 0x310d)
[strip] removed type=48 owner=example.com (txid 0x4b4f)
[strip] removed type=48 owner=example.com (txid 0x4b4f)
[strip] removed type=48 owner=example.com (txid 0xe8af)
[strip] removed type=48 owner=example.com (txid 0xe8af)
[strip] removed type=48 owner=example.com (txid 0xd91f)
[strip] removed type=48 owner=example.com (txid 0xd91f)
[strip] removed type=48 owner=example.com (txid 0x7b5e)
[strip] removed type=48 owner=example.com (txid 0x7b5e)
[attacker] resolver MAC changed -> 5a:3b:93:df:a4:92
[strip+forge] example.com A -> rogue-web (txid 0x3943)
[strip] removed type=46 owner=example.com (txid 0x3943)
[strip] removed type=46 owner=example.com (txid 0x3943)
[attacker] resolver MAC changed -> 5e:f9:55:61:31:a7
[attacker] resolver MAC changed -> da:e3:d0:ab:70:20
[attacker] resolver MAC changed -> 42:96:46:51:77:bc
[strip+forge] example.com A -> rogue-web (txid 0x0251)
[strip] removed type=46 owner=example.com (txid 0x0251)
[strip] removed type=46 owner=example.com (txid 0x0251)
[strip] removed type=48 owner=example.com (txid 0x93f3)
[strip] removed type=48 owner=example.com (txid 0x93f3)
[strip] removed type=48 owner=example.com (txid 0xce7c)
[strip] removed type=48 owner=example.com (txid 0xce7c)
jay-sonawane@JayOmen16:~/network-security/phase4$
```

```
jay-sonawane@JayOmen16:~/network-security/phase4$ docker exec client dig +dnssec example.com

; <<>> DiG 9.18.39-0ubuntu0.24.04.8-Ubuntu <<>> +dnssec example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 48145
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags: do; udp: 1232
; COOKIE: 6934c6023f741af5010000006adb50b82eac31dee171f43e (good)
;; QUESTION SECTION:
;example.com.                   IN      A

;; ANSWER SECTION:
example.com.            3600    IN      A       172.16.1.20
example.com.            3600    IN      RRSIG   A 13 2 3600 20261122111849 20261023111849 4872 example.com. maeakg2urKrqZvU
XZ3wUXyociIj9qJkavzLNfIuo5j460/Qv9yQu1ypw vjntIKUyIUnJ53QQCIvmlhuvAgDt2w==

;; Query time: 11 msec
;; SERVER: 172.16.2.10#53(172.16.2.10) (UDP)
;; WHEN: Fri Oct 23 12:19:04 UTC 2026
;; MSG SIZE  rcvd: 191

jay-sonawane@JayOmen16:~/network-security/phase4$
```

Report answers (§4.4).

- Run A — why can't stripping alone make a zone look unsigned? Signed-ness is not a bit an attacker can flip by deleting bytes; it is something the parent has already told the resolver to expect, via a DS the resolver already saw (directly or cached). Genuinely downgrading to "insecure" requires the parent to supply a signed proof that no DS exists — a real cryptographic artifact the attacker cannot produce without com's private key. Deleting bytes without that proof leaves a response internally inconsistent with what the resolver already expects, which validators treat as an attack signal (bogus/SERVFAIL), not as license to skip validation.
- Run B — what single change reopens the attack, and why? Never publishing example.com's DS in com. This is a real, persistent misconfiguration of the zone, not anything the attacker does on the wire. With no DS, com's genuine signing produces a validly-signed NSEC proving no DS RRset exists at the delegation point; the resolver sees that proof, correctly concludes example.com is a deliberate island outside the chain of trust, and never attempts to validate anything under it — at which point Phase 2-style forgery works again, because there is no validation left to defeat.
- Where does downgrade resistance actually come from? Not from the child zone's own signing, but from the parent's affirmative, cryptographically signed assertion of the DS RRset's status (present with this digest, or provably absent). The child cannot assert its own security; only the parent, as the already-trusted link in the chain, can.
- What is the exact operator mistake that forfeits it? Signing a zone and never submitting or publishing its DS record to the parent. The zone is cryptographically sound in isolation but sits entirely outside the chain of trust — indistinguishable, from the resolver's point of view, from a zone that was never signed at all.

Phase 5 — Zone Walking

Builds on Phase 3's signed, validating deployment. domain-ns's zone gains 5 "secret" names never advertised anywhere else (`backup`, `staging`, `monitor`, `admin-panel`, `internal-vpn`), discoverable only by walking the denial-of-existence chain. The attacker no longer runs an on-path interceptor — there is nothing to intercept; the vulnerability lives in what an ordinary, individually-addressed query is allowed

to return. `core-router` reverts to a plain router; the attacker just queries domain-ns directly like any client would.

Design choices. `scripts/sign-zones.sh` signs example.com with plain NSEC by default (walkable) or NSEC3 with `--nsec3` (the fix), deliberately using the cheapest possible NSEC3 parameters — no salt, zero extra hash iterations — to isolate exactly what the fix buys: walkability, not denial-of-existence itself, which both modes provide equally.

Verification evidence. `dig +dnssec` against a nonexistent name returns NXDOMAIN with the `ad` flag and, in NSEC mode, an NSEC record whose two name fields bracket where the query would have sat alphabetically (`verify_nsec_denial.png`); in NSEC3 mode the equivalent denial uses a hashed owner name instead (`verify_nsec3_denial.png`), confirming authenticated denial still works — only its walkability changed.

Attack evidence. Against the NSEC deployment, `zone_walker` recovers all 7 real owner names (the 5 secret names plus `domain-ns` and `www`) well within the `names+2` query budget (`walker_nsec_success.png`); the chain-walk traffic is shown in `wireshark_nsec_leak.png`. Against the NSEC3 deployment, the identical walker halts on the very first query, reporting a hashed next-owner it cannot use, recovering zero names (`walker_nsec3_blocked.png`).

Screenshots.

```
jay-sonawane@JayOmen16:~/network-security/phase5$ docker exec client dig +dnssec nonexistent-name.example.com

; <<>> DiG 9.18.39-0ubuntu0.24.04.8-Ubuntu <<>> +dnssec nonexistent-name.example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NXDOMAIN, id: 29508
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 0, AUTHORITY: 6, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags: do; udp: 1232
; COOKIE: 6f68244dd518debc010000006adb5cdd4db409573443bedf (good)
;; QUESTION SECTION:
;nonexistent-name.example.com.  IN      A

;; AUTHORITY SECTION:
example.com.            3600    IN      SOA     domain-ns.example.com. admin.example.com. 2026101401 3600 900 604800 3600
example.com.            3600    IN      RRSIG   SOA 13 2 3600 20261122120948 20261023120948 29723 example.com. 5rnYQuOM/1RZ
Vi17KxOKl7xhFdDZYN/9zYu3nmnomL2KCUeokGody8jZ 5QkxaKUMKkYWfUVCiOOK6WnGG8VDXw==
example.com.            3600    IN      RRSIG   NSEC 13 2 3600 20261122120948 20261023120948 29723 example.com. pRVaiuYCEff
hc/MSCFP7/kvR28+njj15LOcY41gXaJIq7tUFJENEuCkC ycUh2FLMmf8q+0Nhwxro5UgQwrkHxg==
example.com.            3600    IN      NSEC    admin-panel.example.com. A NS SOA RRSIG NSEC DNSKEY
monitor.example.com.    3600    IN      RRSIG   NSEC 13 3 3600 20261122120948 20261023120948 29723 example.com. KE1cQ7dHG0S
McHH+rcd8bek08pzIjYljkd6Kkx1c2d1ZzwXeLC+pzTxq n1UDIR8FGOfa8RDiHjzXaW4SZ4nJPA==
monitor.example.com.    3600    IN      NSEC    staging.example.com. A RRSIG NSEC

;; Query time: 8 msec
;; SERVER: 172.16.2.10#53(172.16.2.10) (UDP)
;; WHEN: Fri Oct 23 13:10:53 UTC 2026
;; MSG SIZE  rcvd: 553
```

```
jay-sonawane@JayOmen16:~/network-security/phase5$ docker exec client dig +dnssec nonexistent-name.example.com

; <<>> DiG 9.18.39-0ubuntu0.24.04.8-Ubuntu <<>> +dnssec nonexistent-name.example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NXDOMAIN, id: 39094
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 0, AUTHORITY: 8, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags: do; udp: 1232
; COOKIE: a59e27e01854d42f010000006adb5e497e222cd813ec6a1a (good)
;; QUESTION SECTION:
;nonexistent-name.example.com.  IN      A

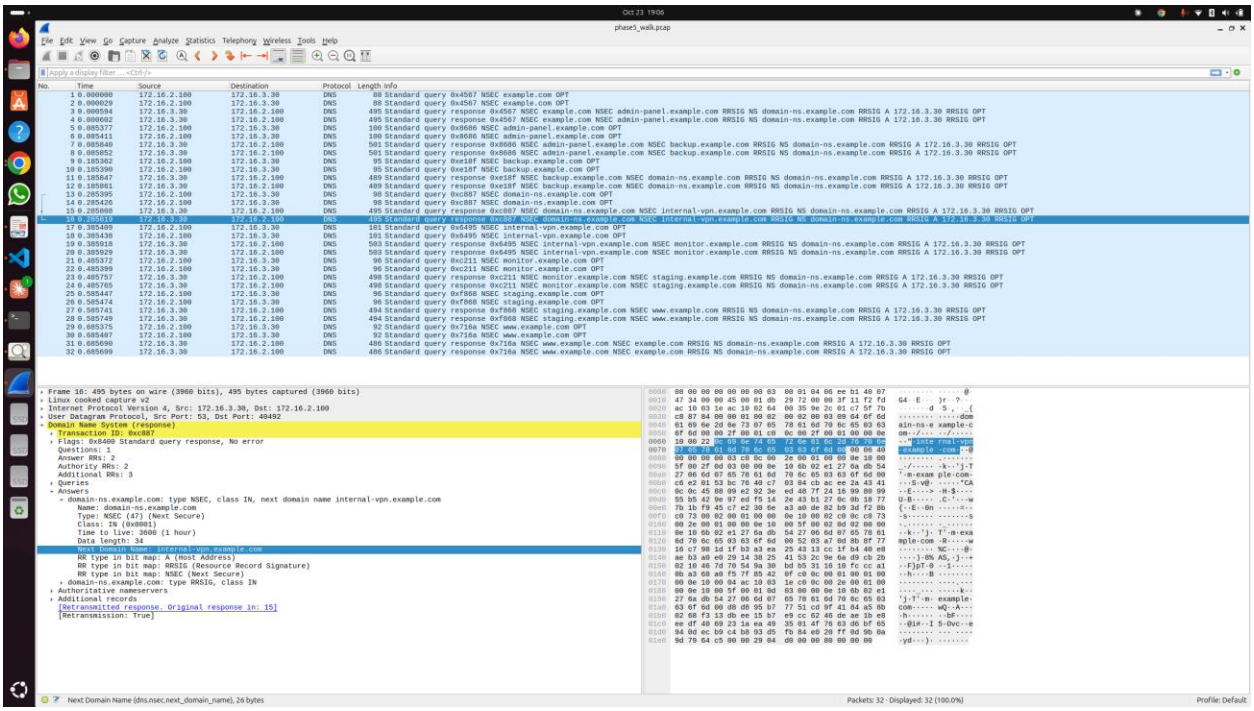
;; AUTHORITY SECTION:
example.com.            3600    IN      SOA     domain-ns.example.com. admin.example.com. 2026101401 3600 900 604800 3600
example.com.            3600    IN      RRSIG   SOA 13 2 3600 20261122121321 20261023121321 24336 example.com. f5VJqBYoeU5d
mjz+dAoDQ6LWp5STixbVbBY1y8XxEljH3QrGbnSk3vIO 2EwwGaWLfOrwlMOtzzFLpLc1jb2nuQ==
1HV0282B4CRSBNF4UBGQECO0LLF35MR1.example.com. 3600 IN RRSIG NSEC3 13 3 3600 20261122121321 20261023121321 24336 example.com
. cih9Za+cLOpZm+q1TO7yfRlxKLR7grA3/E1RWyF9+1y7rgTyM5SP1hEW 4fHpF/po0KPyvA9kIsnhg6Xi9GAFYg==
1HV0282B4CRSBNF4UBGQECO0LLF35MR1.example.com. 3600 IN NSEC3 1 0 0 - 7CBC5T65DEN3RTLEFTLU68F61U5GUR4P A RRSIG
7CBC5T65DEN3RTLEFTLU68F61U5GUR4P.example.com. 3600 IN RRSIG NSEC3 13 3 3600 20261122121321 20261023121321 24336 example.com
. uX3BGgMVN9fZIXC6O8e0ETvwe+pyHAfZn2kUDGnc1ZZsliTJ1ltNT4Nj yTup8v21819+1IrVwCmPGEVzpP6T2A==
7CBC5T65DEN3RTLEFTLU68F61U5GUR4P.example.com. 3600 IN NSEC3 1 0 0 - ESFG72OS2E7U804JP8QLFRDDCC74VESS A RRSIG
ONIB9MGUB9H0RML3CDF5BGRJ59DKJHVK.example.com. 3600 IN RRSIG NSEC3 13 3 3600 20261122121321 20261023121321 24336 example.com
. toU+Wd9Xv4jec8k0MmUiABiKqPmLBTRiKGHYZJKhNIGLnJ7YMSAAgoGV LuZX6vsGkumZL0/RfnQSH7iq0/ID0w==
ONIB9MGUB9H0RML3CDF5BGRJ59DKJHVK.example.com. 3600 IN NSEC3 1 0 0 - VGIPM86MRLGTR7D21T3O4OQEF61DPOK9 A NS SOA RRSIG DNSKEY
NSEC3PARAM

;; Query time: 9 msec
;; SERVER: 172.16.2.10#53(172.16.2.10) (UDP)
;; WHEN: Fri Oct 23 13:16:57 UTC 2026
;; MSG SIZE  rcvd: 803
```

```
jay-sonawane@JayOmen16:~/network-security/phase5$ docker exec attacker zone_walker eth0
=== Phase 5 zone walker ===
Target server: 172.16.3.30   Zone apex: example.com.   Next hop: 172.16.2.3
Method: direct QTYPE=NSEC queries, following the next-domain-name chain. No AXFR, no wordlist.

[1] Query NSEC for example.com.
    Next name in zone: admin-panel.example.com.
[2] Query NSEC for admin-panel.example.com.
    Next name in zone: backup.example.com.
[3] Query NSEC for backup.example.com.
    Next name in zone: domain-ns.example.com.
[4] Query NSEC for domain-ns.example.com.
    Next name in zone: internal-vpn.example.com.
[5] Query NSEC for internal-vpn.example.com.
    Next name in zone: monitor.example.com.
[6] Query NSEC for monitor.example.com.
    Next name in zone: staging.example.com.
[7] Query NSEC for staging.example.com.
    Next name in zone: www.example.com.
[8] Query NSEC for www.example.com.
    Next name in zone: example.com.

Walk complete.
Recovered 7 name(s):
  admin-panel.example.com.
  backup.example.com.
  domain-ns.example.com.
  internal-vpn.example.com.
  monitor.example.com.
  staging.example.com.
  www.example.com.
Total distinct owner names (including apex): 8
Queries used: 8   Assignment limit (names+2): 10   PASS
```



```
jay-sonawane@JayOmen16:~/network-security/phase5$ docker exec attacker zone_walker eth0
=== Phase 5 zone walker ===
Target server: 172.16.3.30   Zone apex: example.com.   Next hop: 172.16.2.3
Method: direct QTYPE=NSEC queries, following the next-domain-name chain. No AXFR, no wordlist.

[1] Query NSEC for example.com.
    Received NSEC3: the next-owner field is a hash, not a readable name.

Walk blocked after 1 query. Plaintext names recovered: 0.
jay-sonawane@JayOmen16:~/network-security/phase5$
```

Report answers (§5.4).

- Which names were recovered, and in how many steps? All 5 secret names plus `domain-ns` and `www` (7 total), each learned from the "next owner" field of the previous NSEC response, within the `names+2` query budget — the walker's own printed trace of each query and the name it learned is the primary evidence here.
- Which configuration made the walk possible, and why does signed denial leak what plain DNS doesn't? Plain NSEC signing. Plain DNS answers a nonexistent-name query with a direct, computed-on-the-spot NXDOMAIN that says nothing about any other name. A signed denial cannot work that way, because the signature has to be produced in advance, before anyone knows which nonexistent name will eventually be asked about — so what gets signed instead is a standing statement about the zone's actual contents ("nothing exists between this real name and that real name"), reused for every query that falls in the gap. That statement is only useful if it names real, existing owners — exactly the information plain DNS never had to disclose.
- What does the NSEC3 fix cost the operator, and what still leaks afterward? Extra hashing cost at signing time, and extra validation cost if a nonzero iteration count or salt is added (deliberately omitted here to isolate the point). What an enumerator can still learn: the zone's size (the count of NSEC3 records is still visible even with names hashed), and, offline, any name short or

guessable enough to be found by dictionary/brute-force hashing against the exposed hash values — salting and a higher iteration count raise that cost but do not eliminate it.

Phase 6 — Last-Mile Response Forgery

Phase 3's signed, validating hierarchy upstream is completely untouched — root-ns/tld-ns/domain-ns still sign, the resolver still validates with its own trust anchor. What changes: the attacker relocates from the resolver subnet (172.16.2.100) to the **client subnet** (172.16.1.100), moving to the one hop DNSSEC never covers — the resolver-to-stub link. The diversion route moves correspondingly, from core-router to edge-router (`172.16.1.10/32 via 172.16.1.100`, resolver→client replies only). The client gains its own `trust-anchor.conf` (the same root KSK the resolver trusts) so it can run `delv` as a locally-validating client, independent of whatever the resolver's reply claims. The attacker also unconditionally sets the `ad` bit on every forged packet, to make a concrete point about that bit's trustworthiness.

Design choices. Same forging mechanics as Phase 2 (in-place A-record RDATA overwrite), simply relocated to a different link — the point of this phase is topological, not a new interceptor design.

Attack evidence. Client A, an ordinary stub, is fully deceived: `dig example.com` returns `172.16.1.30` with the `ad` flag set, and `curl` fetches the defaced page (`clientA_dig.png`, `clientA_curl.png`, `clientA_forged_response.png`). Client B, running `delv` with its own trust anchor over the identical compromised wire path, fails validation (`clientB_delv_fail.png`, `clientB_rrsig_mismatch.png`) — it never looks at the `ad` bit at all, independently re-fetching the RRSIG/DNSKEY data and re-deriving the signature check itself. The attacker's log and the full exchange capture are in `attacker_log.png` and `full_exchange.png`; verification that upstream DNSSEC is healthy before the attack is in `verify_ad_flag.png`, `verify_curl.png`, `verify_delv.png`, `verify_dig.png`.

Screenshots.



```
jay-sonawane@JayOmen16:~/network-security/phase6$ docker exec client curl -s http://example.com
<!DOCTYPE html>
<html><head><title>example.com</title></head>
<body>
  <h1>This is the REAL example.com</h1>
  <p>Served by webserver (172.16.1.20).</p>
</body></html>
jay-sonawane@JayOmen16:~/network-security/phase6$
```

```
jay-sonawane@JayOmen16:~/network-security/phase6$ docker exec client delv @172.16.2.10 -a /etc/bind/trust-anchor.conf examp
le.com
; fully validated
example.com.            3431    IN      A       172.16.1.20
example.com.            3431    IN      RRSIG   A 13 2 3600 20261122125632 20261023125632 29963 example.com. Zxz/qciNxwMW65
c1u68D0JO8IyMMBLQLJQHA2eaha7tAjFzzjv0hFG4l fTOkI5McWthp7Nw9hYC6oUc83aI6eA==
jay-sonawane@JayOmen16:~/network-security/phase6$
```

```
jay-sonawane@JayOmen16:~/network-security/phase6$ docker exec client dig example.com

; <<>> DiG 9.18.39-0ubuntu0.24.04.8-Ubuntu <<>> example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 2601
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: e7c3c854f36ca5cf010000006adb6c0f0371b40fed9a0765 (good)
;; QUESTION SECTION:
;example.com.                   IN      A

;; ANSWER SECTION:
example.com.            2893    IN      A       172.16.1.30

;; Query time: 0 msec
;; SERVER: 172.16.2.10#53(172.16.2.10) (UDP)
;; WHEN: Fri Oct 23 14:15:43 UTC 2026
;; MSG SIZE  rcvd: 84
```

```
jay-sonawane@JayOmen16:~/network-security/phase6$ docker exec client dig example.com

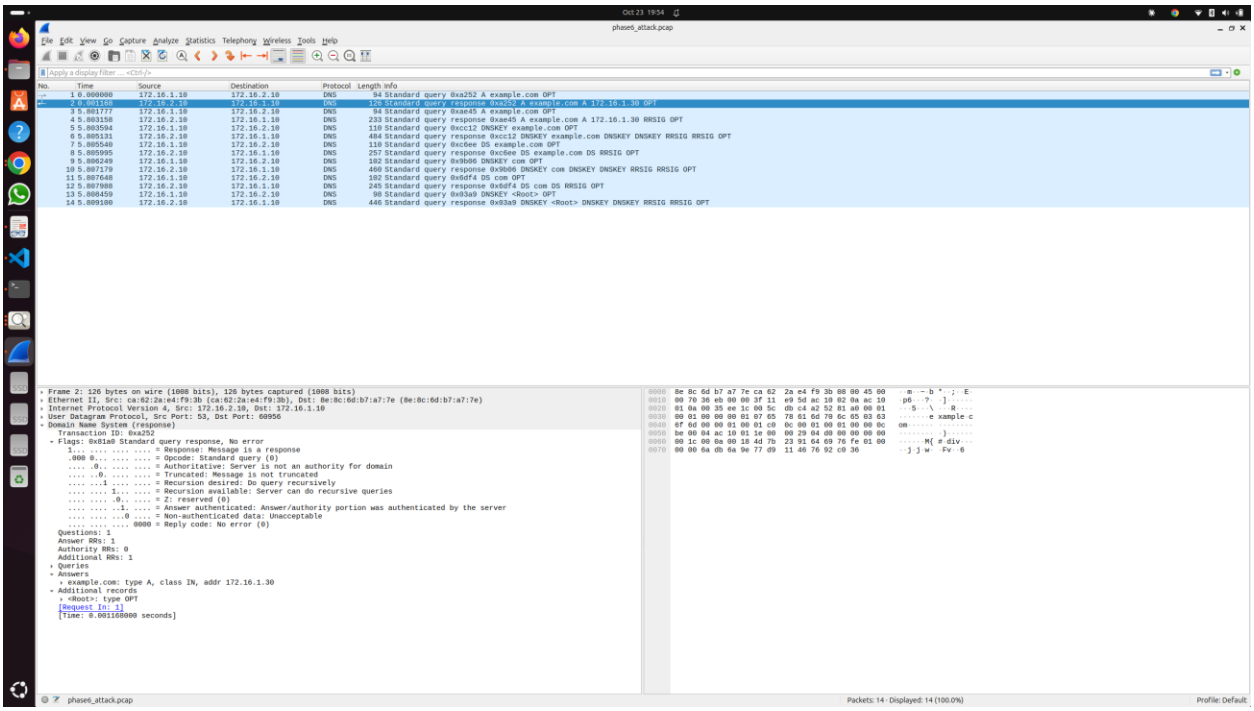
; <<>> DiG 9.18.39-0ubuntu0.24.04.8-Ubuntu <<>> example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 2601
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: e7c3c854f36ca5cf010000006adb6c0f0371b40fed9a0765 (good)
;; QUESTION SECTION:
;example.com.                   IN      A

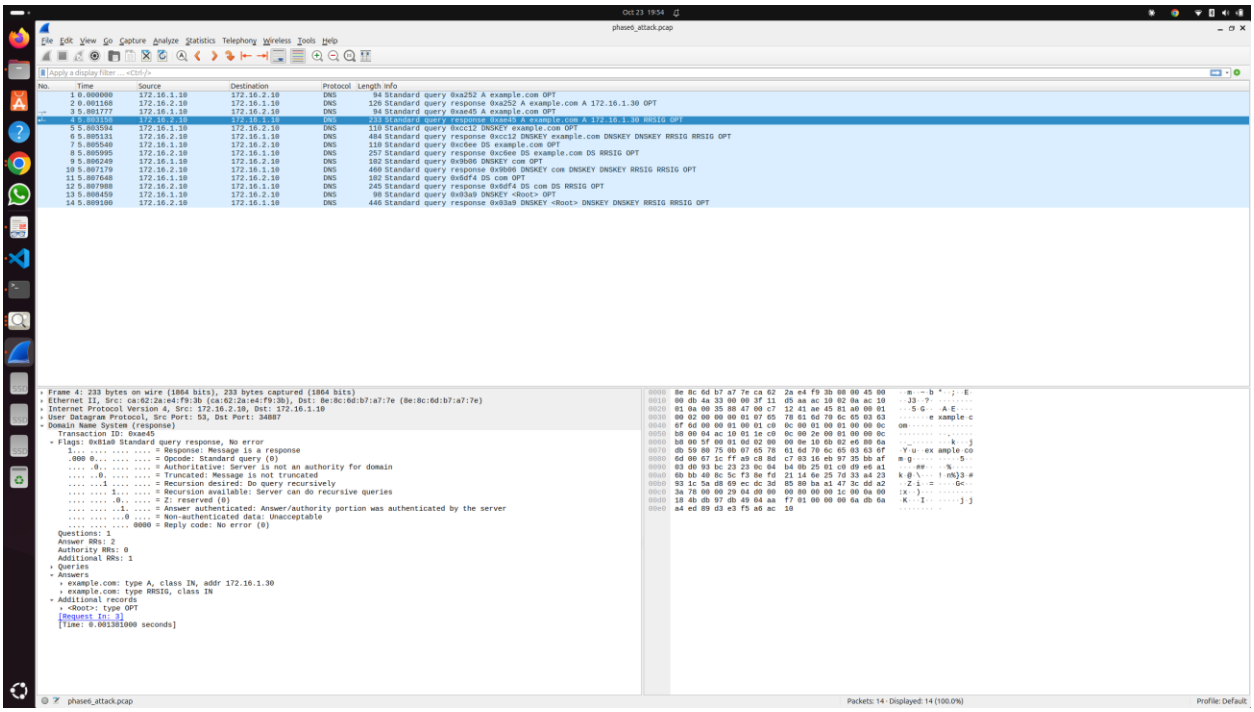
;; ANSWER SECTION:
example.com.            2893    IN      A       172.16.1.30

;; Query time: 0 msec
;; SERVER: 172.16.2.10#53(172.16.2.10) (UDP)
;; WHEN: Fri Oct 23 14:15:43 UTC 2026
;; MSG SIZE  rcvd: 84
```

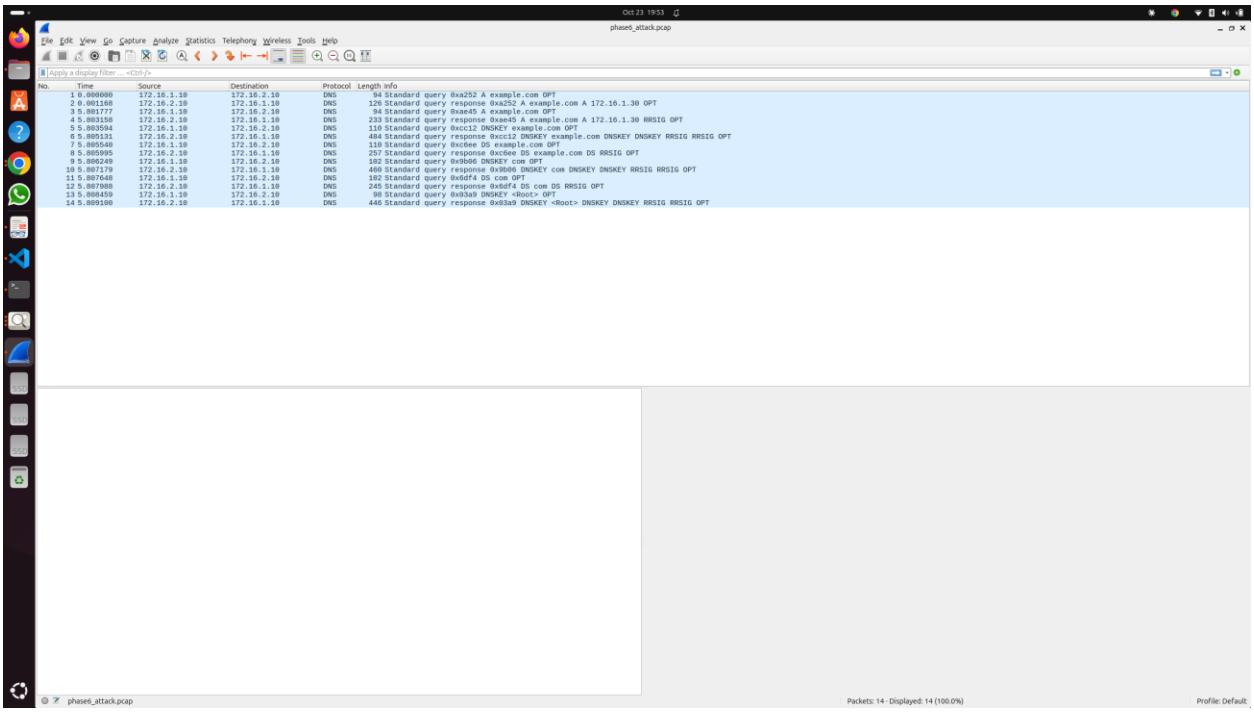
```
jay-sonawane@JayOmen16:~/network-security/phase6$ docker exec client curl -s http://example.com
<!DOCTYPE html>
<html><head><title>example.com</title></head>
<body style="background:#2b0000;color:#fff;font-family:sans-serif;text-align:center;padding-top:80px;">
  <h1>DEFACED</h1>
  <p>This is rogue-web (172.16.1.30). example.com just got cache-poisoned.</p>
</body></html>
jay-sonawane@JayOmen16:~/network-security/phase6$
```




```
jay-sonawane@JayOmen16:~/network-security/phase6$ docker exec client delv @172.16.2.10 -a /etc/bind/trust-anchor.conf example.com
;; validating example.com/A: no valid signature found
;; RRSIG failed to verify resolving 'example.com/A/IN': 172.16.2.10#53
;; resolution failed: RRSIG failed to verify
jay-sonawane@JayOmen16:~/network-security/phase6$
```



```
jay-sonawane@JayOmen16:~/network-security/phase6$ docker logs attacker --tail 30
sysctl: setting key "net.ipv4.ip_forward", ignoring: Read-only file system
net.ipv4.ip_forward = 0
[attacker] priming ARP for the real client (172.16.1.10)...
[attacker] client MAC = 8e:8c:6d:b7:a7:7e
[attacker] up on eth0, target=example.com, rogue=172.16.1.30, client=172.16.1.10, forging=OFF
[forge] example.com A -> rogue-web, ad=1 (txid 0x032a)
[forge] example.com A -> rogue-web, ad=1 (txid 0x5040)
[forge] example.com A -> rogue-web, ad=1 (txid 0xbb00)
[forge] example.com A -> rogue-web, ad=1 (txid 0xdbc8)
[forge] example.com A -> rogue-web, ad=1 (txid 0xac4e)
[forge] example.com A -> rogue-web, ad=1 (txid 0xb79b)
[forge] example.com A -> rogue-web, ad=1 (txid 0xa3c1)
[forge] example.com A -> rogue-web, ad=1 (txid 0xe89c)
[forge] example.com A -> rogue-web, ad=1 (txid 0x4695)
[forge] example.com A -> rogue-web, ad=1 (txid 0xc09a)
[forge] example.com A -> rogue-web, ad=1 (txid 0xa252)
[forge] example.com A -> rogue-web, ad=1 (txid 0xae45)
[forge] example.com A -> rogue-web, ad=1 (txid 0x0a29)
[forge] example.com A -> rogue-web, ad=1 (txid 0x1fa9)
[forge] example.com A -> rogue-web, ad=1 (txid 0x580e)
jay-sonawane@JayOmen16:~/network-security/phase6$
```



Report answers (§6.4).

- What is the outcome for each client? Client A is fully deceived — resolves to `172.16.1.30`, fetches the defaced page, and the forged response even carries a meaningless, self-asserted `ad` flag. Client B fails validation on the exact same wire path, because it never trusts the `ad` bit in the first place.
- Why doesn't upstream DNSSEC stop this? Validation and trust placement are two different things that happen to coincide at the resolver in this topology, but don't have to. The resolver does validate — against the root key it holds as its own trust anchor. But the client never validates anything itself; it just trusts that whatever UDP packet shows up claiming to be from 172.16.2.10 is the resolver's genuine, already-validated answer. DNSSEC's guarantee ends at the last system that actually checks a signature — the resolver, not the client. Everything

downstream of that is exactly as vulnerable as Phase 2, because nothing downstream was ever covered by the signatures in the first place.

- What does the `ad` flag mean, and why can an attacker set it at will? `ad` is one unauthenticated, unsigned header bit, mutable by anyone who can touch the packet, exactly like every other header bit. It is meant as a hint from a resolver to a client that says "I checked this already" — useful only if the channel carrying that hint is itself trusted (a loopback resolver, or an encrypted/authenticated transport — see Phase 7). Over a plain, unauthenticated UDP hop it is just a claim, demonstrated directly here by the interceptor setting it on data it fabricated itself.
- What does Client B do differently that catches the forgery? Not reading the `ad` bit more carefully — not trusting it at all. `delv` re-fetches the actual DNSSEC records and re-runs the cryptographic check locally, which the attacker cannot pass without a private key it does not have.

Phase 7 — Encrypting the Last Mile (DoT)

Reuses Phase 6's deployment. The resolver now also listens for DNS-over-TLS on port 853 (plain 53 stays up too, deliberately, since attempt (iii) needs a cleartext door to test blocking against). A private CA issues a resolver certificate whose DNS SAN is exactly `resolver.cs6008.local`, no IP SAN; the client reaches the resolver by that name (`extra_hosts`) and trusts the CA via its real system trust store — no `-k`/insecure bypass anywhere in either client script. Two client postures, both thin `kdig` wrappers: `query-strict.sh` (DoT only, no fallback) and `query-permissive.sh` (DoT first, falls back to cleartext `dig` on any failure).

Design choices. Edge-router's diversion route is unchanged from Phase 6 (resolver→client only), used by attempts (i) and (iii). Attempt (ii) (impersonation) does not touch edge-router at all: true on-path TCP interception for that direction was attempted and abandoned — the attacker has no interface on `resolver_net`, so relaying a client→resolver packet onward means handing it back to the only router the attacker has, which just re-applies its own diversion rule and loops forever; a DNAT-based variant hit real hairpin-NAT conntrack issues specific to this topology that were not worth chasing further for what they would actually prove. Instead, attempt (ii) is tested by pointing the client's connection directly at the attacker's address for one query while still checking the certificate against the real hostname (`resolver.cs6008.local`) — this tests exactly the same certificate-validation property a true on-path redirect would. The attacker also gains a second program, `tls_endpoint` (OpenSSL and plain POSIX sockets, not libpcap): completing a cryptographically valid TLS handshake requires a real TLS stack negotiating real key material, which cannot be accomplished by crafting raw bytes the way every other attack program in this assignment does.

Verification evidence. `query-strict.sh example.com` returns a clean answer over DoT, `172.16.1.20`, validated against `resolver.cs6008.local` with no bypass flags (`verify_dot_success.png`); upstream DNSSEC is independently confirmed still healthy via `delv` (`verify_delv.png`); and a tcpdump capture during the query shows TCP/853 traffic only, no UDP/53 at all (`verify_no_cleartext.png`).

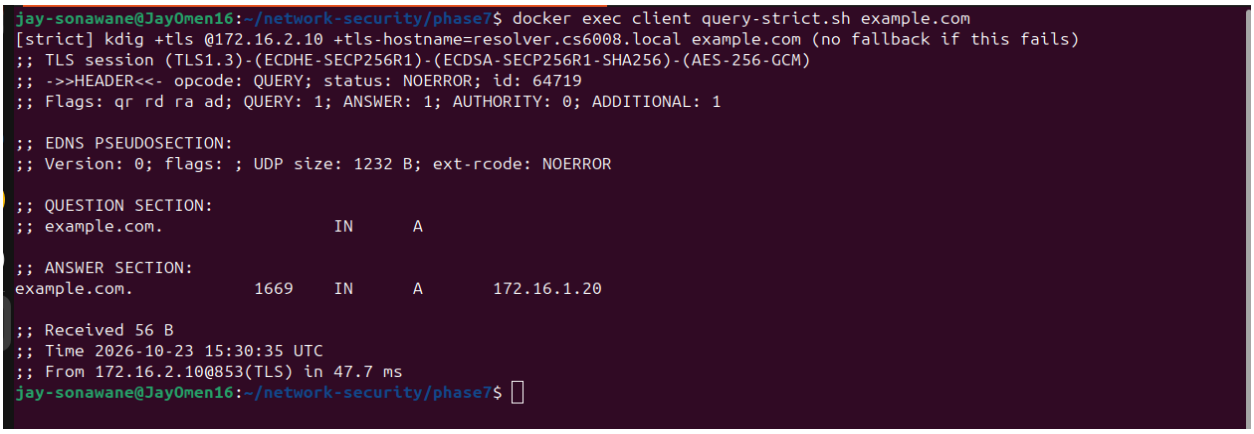
Attack evidence — the six outcomes. `forge.on` is enabled throughout so that whenever a permissive client does fall back to cleartext, the fallback lands on the forged answer rather than the real one.

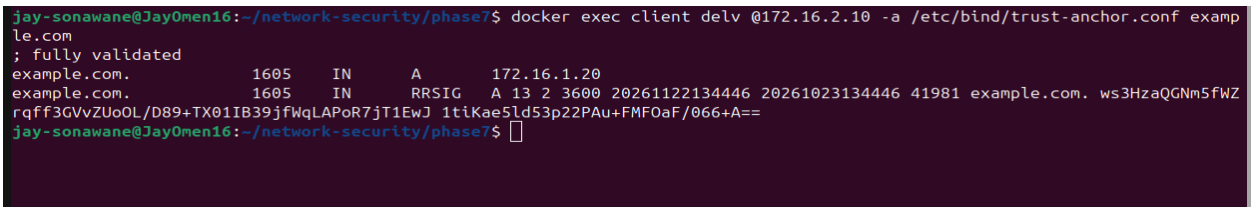
- (i) Alter the answer in transit: strict client fails — the TLS integrity check catches the tamper (`attackI_strict_fail.png`); permissive client's DoT fails the same way, falls back to cleartext, and lands on the forged answer, `172.16.1.30` (`attackI_permissive_lands.png`,

attackI_attacker_log.png). The packet-level evidence is in wireshark_tamper_full.png and wireshark_tamper_forged_answer.png.

- (ii) Impersonate the resolver with a rogue certificate: strict client fails — the rogue cert does not chain to the trusted CA (attackII_strict_fail.png); permissive client rejects the cert identically, falls back to cleartext, and lands on the forged answer (attackII_permissive_lands.png). tls_endpoint's own handshake log is in attackII_attacker_log.png.
- (iii) Deny the encrypted transport: strict client fails — the connection to port 853 times out/is refused (attackIII_strict_fail.png); permissive client times out on DoT, falls back to cleartext, and lands on the forged answer (attackIII_permissive_lands.png).

Screenshots.





```
^Cjay-sonawane@JayOmen16:~/network-security/phase7$ docker exec edge-router timeout 8 tcpdump -i eth0 -nn port 53 or port 3
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
19 packets captured
19 packets received by filter
0 packets dropped by kernel
15:33:15.386050 IP 172.16.1.10.37881 > 172.16.2.10.853: Flags [S], seq 685633836, win 64240, options [mss 1460,sackOK,TS va
l 1527935956 ecr 0,nop,wscale 10], length 0
15:33:15.386082 IP 172.16.2.10.853 > 172.16.1.10.37881: Flags [S.], seq 1308284678, ack 685633837, win 65232, options [mss
1220,sackOK,TS val 209845596 ecr 1527935956,nop,wscale 10], length 0
15:33:15.386278 IP 172.16.1.10.37881 > 172.16.2.10.853: Flags [.], ack 1, win 63, options [nop,nop,TS val 1527935957 ecr 20
9845596], length 0
15:33:15.386621 IP 172.16.1.10.37881 > 172.16.2.10.853: Flags [P.], seq 1:410, ack 1, win 63, options [nop,nop,TS val 15279
35957 ecr 209845596], length 409
15:33:15.386651 IP 172.16.2.10.853 > 172.16.1.10.37881: Flags [.], ack 410, win 64, options [nop,nop,TS val 209845596 ecr 1
527935957], length 0
15:33:15.387653 IP 172.16.2.10.853 > 172.16.1.10.37881: Flags [P.], seq 1:939, ack 410, win 64, options [nop,nop,TS val 209
845597 ecr 1527935957], length 938
15:33:15.388270 IP 172.16.1.10.37881 > 172.16.2.10.853: Flags [.], ack 939, win 63, options [nop,nop,TS val 1527935959 ecr
209845597], length 0
15:33:15.388677 IP 172.16.1.10.37881 > 172.16.2.10.853: Flags [P.], seq 410:416, ack 939, win 63, options [nop,nop,TS val 1
527935959 ecr 209845597], length 6
15:33:15.390392 IP 172.16.1.10.37881 > 172.16.2.10.853: Flags [P.], seq 416:490, ack 939, win 63, options [nop,nop,TS val 1
527935961 ecr 209845597], length 74
15:33:15.390469 IP 172.16.2.10.853 > 172.16.1.10.37881: Flags [.], ack 490, win 64, options [nop,nop,TS val 209845600 ecr 1
527935959], length 0
15:33:15.390487 IP 172.16.1.10.37881 > 172.16.2.10.853: Flags [P.], seq 490:642, ack 939, win 63, options [nop,nop,TS val 1
527935961 ecr 209845597], length 152
15:33:15.390746 IP 172.16.2.10.853 > 172.16.1.10.37881: Flags [P.], seq 939:1481, ack 642, win 64, options [nop,nop,TS val
209845600 ecr 1527935961], length 542
15:33:15.432262 IP 172.16.1.10.37881 > 172.16.2.10.853: Flags [.], ack 1481, win 63, options [nop,nop,TS val 1527936003 ecr
 209845600], length 0
15:33:15.432309 IP 172.16.2.10.853 > 172.16.1.10.37881: Flags [P.], seq 1481:1561, ack 642, win 64, options [nop,nop,TS val
 209845642 ecr 1527936003], length 80
15:33:15.433430 IP 172.16.1.10.37881 > 172.16.2.10.853: Flags [.], ack 1561, win 63, options [nop,nop,TS val 1527936004 ecr
 209845642], length 0
15:33:15.433799 IP 172.16.1.10.37881 > 172.16.2.10.853: Flags [P.], seq 642:666, ack 1561, win 63, options [nop,nop,TS val
1527936004 ecr 209845642], length 24
15:33:15.433853 IP 172.16.1.10.37881 > 172.16.2.10.853: Flags [F.], seq 666, ack 1561, win 63, options [nop,nop,TS val 1527
936004 ecr 209845642], length 0
15:33:15.434029 IP 172.16.2.10.853 > 172.16.1.10.37881: Flags [F.], seq 1561, ack 667, win 64, options [nop,nop,TS val 2098
45643 ecr 1527936004], length 0
15:33:15.434199 IP 172.16.1.10.37881 > 172.16.2.10.853: Flags [.], ack 1562, win 63, options [nop,nop,TS val 1527936005 ecr
 209845643], length 0

jay-sonawane@JayOmen16:~/network-security/phase7$
```

```
jay-sonawane@JayOmen16:~/network-security/phase7$ docker exec attacker touch /tmp/forge.on
docker exec attacker touch /tmp/tamper.on
docker exec client query-strict.sh example.com
[strict] kdig +tls @172.16.2.10 +tls-hostname=resolver.cs6008.local example.com (no fallback if this fails)
;; WARNING: TLS, handshake failed (An unexpected TLS packet was received.)
;; ERROR: failed to query server 172.16.2.10@853(TCP)
jay-sonawane@JayOmen16:~/network-security/phase7$
```

```
jay-sonawane@JayOmen16:~/network-security/phase7$ docker exec client query-permissive.sh example.com
[permissive] trying DoT first: kdig +tls @172.16.2.10 +tls-hostname=resolver.cs6008.local example.com
;; WARNING: TLS, handshake failed (An unexpected TLS packet was received.)
;; ERROR: failed to query server 172.16.2.10@853(TCP)
[permissive] DoT failed -- falling back to cleartext UDP/53: dig @172.16.2.10 example.com

; <<>> DiG 9.18.39-0ubuntu0.24.04.8-Ubuntu <<>> @172.16.2.10 example.com
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 61771
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

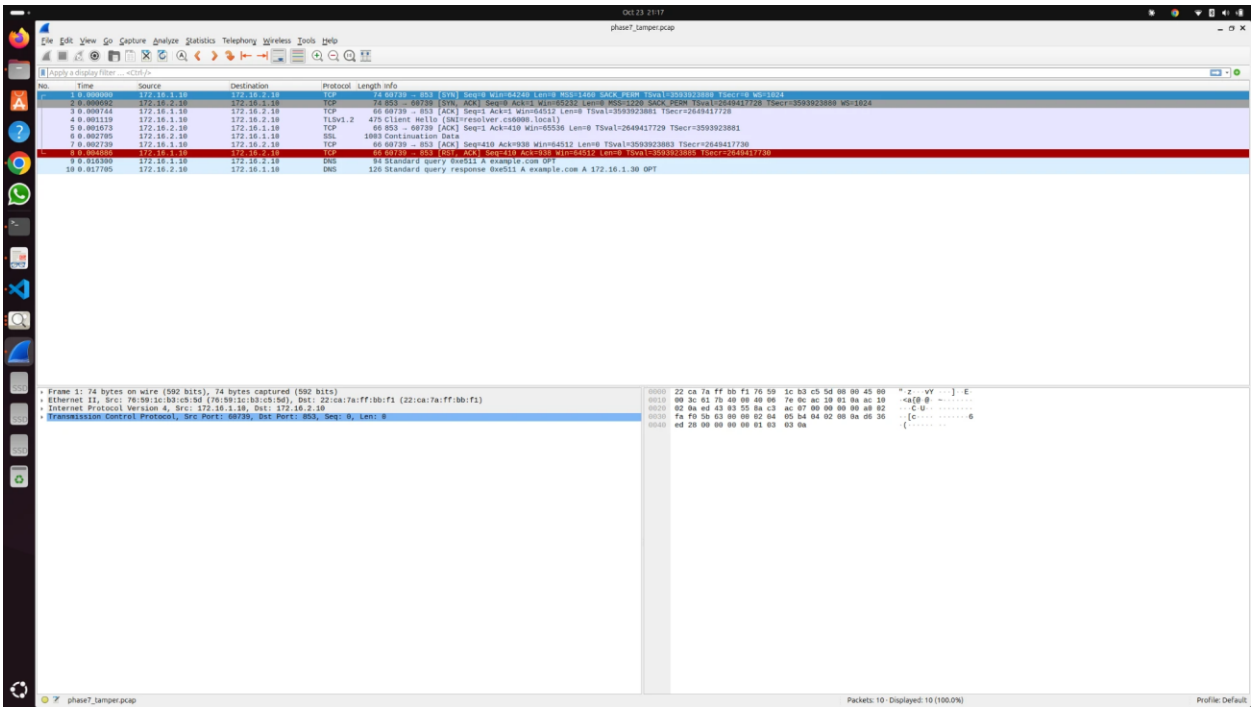
;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 622c7cedbdb95a76010000006adb7ecbfe10b7fd6df8ba62 (good)
;; QUESTION SECTION:
;example.com.                   IN      A

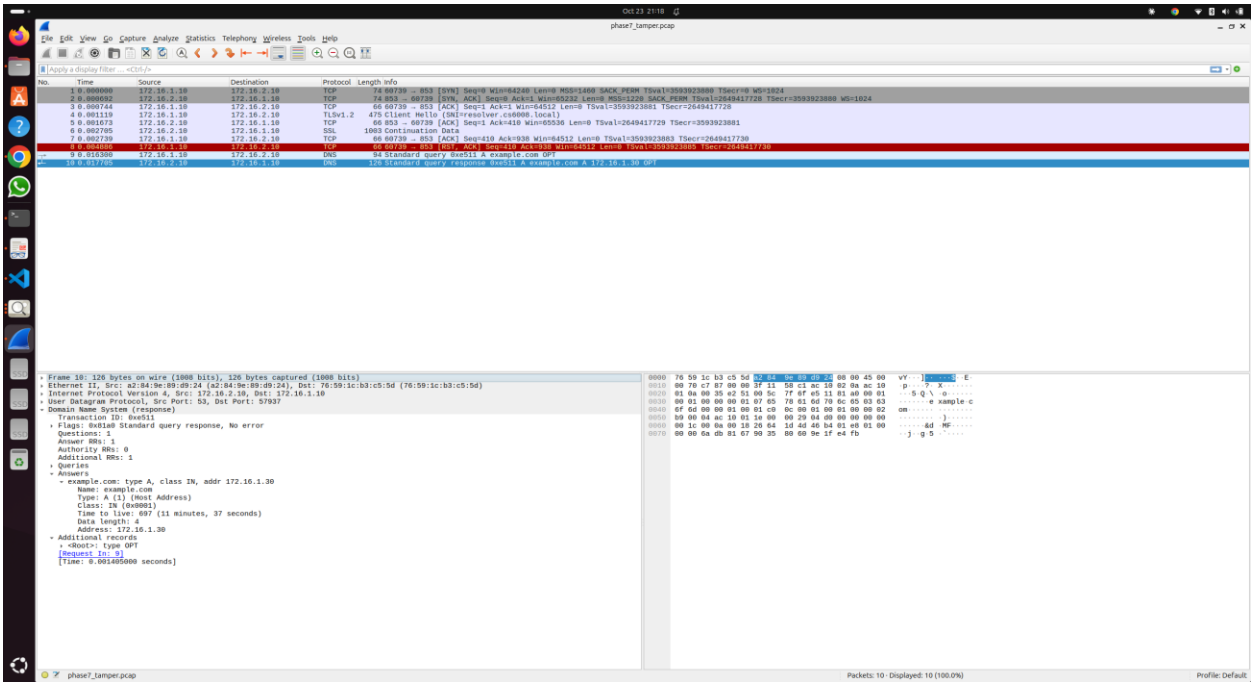
;; ANSWER SECTION:
example.com.            1365    IN      A       172.16.1.30

;; Query time: 0 msec
;; SERVER: 172.16.2.10#53(172.16.2.10) (UDP)
;; WHEN: Fri Oct 23 15:35:39 UTC 2026
;; MSG SIZE  rcvd: 84

jay-sonawane@JayOmen16:~/network-security/phase7$
```

```
jay-sonawane@JayOmen16:~/network-security/phase7$ docker logs attacker --tail 15
[attacker] client MAC = 76:59:1c:b3:c5:5d
[tls_endpoint] listening on 172.16.1.100:853 -- reached by pointing a client at this address directly for §7.3 attempt (ii)
 (see README)
[attacker] up on eth0, target=example.com, rogue=172.16.1.30, client=172.16.1.10
           forging=OFF tamper=OFF deny=OFF
[tls_endpoint] IMPERSONATE: presenting rogue cert for resolver.cs6008.local
[tls_endpoint] handshake rejected by the client (expected for a client that validates the cert chain properly -- our rogue
cert isn't signed by the trusted CA)
[tls_endpoint] IMPERSONATE: presenting rogue cert for resolver.cs6008.local
[tls_endpoint] handshake rejected by the client (expected for a client that validates the cert chain properly -- our rogue
cert isn't signed by the trusted CA)
[tls_endpoint] IMPERSONATE: presenting rogue cert for resolver.cs6008.local
[tls_endpoint] handshake rejected by the client (expected for a client that validates the cert chain properly -- our rogue
cert isn't signed by the trusted CA)
[forge] example.com A -> rogue-web, ad=1 (txid 0x50b1) [cleartext fallback]
[forge] example.com A -> rogue-web, ad=1 (txid 0xb925) [cleartext fallback]
[tamper] flipped a byte in a TCP/853 segment toward the client (seg_len=970) -- expect the TLS session to break
[tamper] flipped a byte in a TCP/853 segment toward the client (seg_len=968) -- expect the TLS session to break
[forge] example.com A -> rogue-web, ad=1 (txid 0xf14b) [cleartext fallback]
jay-sonawane@JayOmen16:~/network-security/phase7$
```






```
jay-sonawane@JayOmen16:~/network-security/phase7$ docker exec attacker rm -f /tmp/tamper.on
jay-sonawane@JayOmen16:~/network-security/phase7$ docker exec client query-strict.sh example.com 172.16.1.100
[strict] kdig +tls @172.16.1.100 +tls-hostname=resolver.cs6008.local example.com (no fallback if this fails)
;; WARNING: TLS, handshake failed (Error in the certificate.)
;; ERROR: failed to query server 172.16.1.100@853(TCP)
jay-sonawane@JayOmen16:~/network-security/phase7$
```

```
jay-sonawane@JayOmen16:~/network-security/phase7$ docker exec client query-permissive.sh example.com 172.16.1.100
[permissive] trying DoT first: kdig +tls @172.16.1.100 +tls-hostname=resolver.cs6008.local example.com
;; WARNING: TLS, handshake failed (Error in the certificate.)
;; ERROR: failed to query server 172.16.1.100@853(TCP)
[permissive] DoT failed -- falling back to cleartext UDP/53: dig @172.16.2.10 example.com

; <<>> DiG 9.18.39-0ubuntu0.24.04.8-Ubuntu <<>> @172.16.2.10 example.com
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 27868
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 181efb888c0a046d010000006adb7f94414c040158f7d314 (good)
;; QUESTION SECTION:
;example.com.                   IN      A

;; ANSWER SECTION:
example.com.            1164    IN      A       172.16.1.30

;; Query time: 0 msec
;; SERVER: 172.16.2.10#53(172.16.2.10) (UDP)
;; WHEN: Fri Oct 23 15:39:00 UTC 2026
;; MSG SIZE  rcvd: 84

jay-sonawane@JayOmen16:~/network-security/phase7$
```

```
jay-sonawane@JayOmen16:~/network-security/phase7$ docker logs attacker --tail 10
[forge] example.com A -> rogue-web, ad=1 (txid 0x50b1) [cleartext fallback]
[forge] example.com A -> rogue-web, ad=1 (txid 0xb925) [cleartext fallback]
[tamper] flipped a byte in a TCP/853 segment toward the client (seg_len=970) -- expect the TLS session to break
[tamper] flipped a byte in a TCP/853 segment toward the client (seg_len=968) -- expect the TLS session to break
[forge] example.com A -> rogue-web, ad=1 (txid 0xf14b) [cleartext fallback]
[tls_endpoint] IMPERSONATE: presenting rogue cert for resolver.cs6008.local
[tls_endpoint] handshake rejected by the client (expected for a client that validates the cert chain properly -- our rogue
cert isn't signed by the trusted CA)
[tls_endpoint] IMPERSONATE: presenting rogue cert for resolver.cs6008.local
[tls_endpoint] handshake rejected by the client (expected for a client that validates the cert chain properly -- our rogue
cert isn't signed by the trusted CA)
[forge] example.com A -> rogue-web, ad=1 (txid 0x6cdc) [cleartext fallback]
jay-sonawane@JayOmen16:~/network-security/phase7$
```

```
jay-sonawane@JayOmen16:~/network-security/phase7$ docker exec attacker touch /tmp/deny.on
docker exec client query-strict.sh example.com
[strict] kdig +tls @172.16.2.10 +tls-hostname=resolver.cs6008.local example.com (no fallback if this fails)
;; WARNING: connection timeout for 172.16.2.10@853(TLS)
;; ERROR: failed to query server 172.16.2.10@853(TCP)
jay-sonawane@JayOmen16:~/network-security/phase7$
```

```
jay-sonawane@JayOmen16:~/network-security/phase7$ docker exec client query-permissive.sh example.com
[permissive] trying DoT first: kdig +tls @172.16.2.10 +tls-hostname=resolver.cs6008.local example.com
;; WARNING: connection timeout for 172.16.2.10@853(TLS)
;; ERROR: failed to query server 172.16.2.10@853(TCP)
[permissive] DoT failed -- falling back to cleartext UDP/53: dig @172.16.2.10 example.com

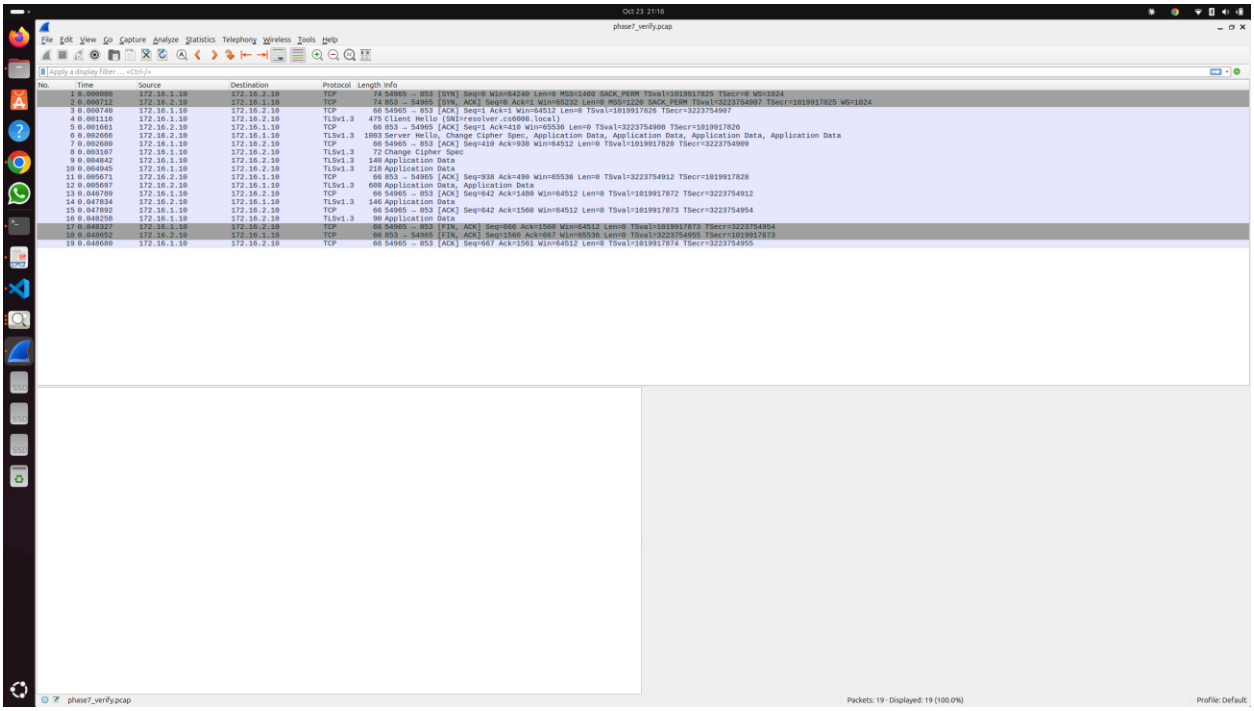
; <<>> DiG 9.18.39-0ubuntu0.24.04.8-Ubuntu <<>> @172.16.2.10 example.com
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 39888
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 7db12b2707f3b736010000006adb8064cdcce763913f3823 (good)
;; QUESTION SECTION:
;example.com.                   IN      A

;; ANSWER SECTION:
example.com.            956     IN      A       172.16.1.30

;; Query time: 0 msec
;; SERVER: 172.16.2.10#53(172.16.2.10) (UDP)
;; WHEN: Fri Oct 23 15:42:28 UTC 2026
;; MSG SIZE  rcvd: 84

jay-sonawane@JayOmen16:~/network-security/phase7$
```



Report answers (§7.4).

- What do the six outcomes show? All three attempts fail directly against a strict client and succeed against a permissive one — but by the same mechanism every time. None of the three ever actually breaks into an established, validly-authenticated TLS session; they all just force that session to not exist (tampering trips the integrity check and the connection dies; impersonation never gets a trusted handshake in the first place; denial just blocks the port outright). The strict client treats that as fatal and stops. The permissive client treats it as "try the other door," and that other door was never protected by anything this phase added — it is exactly Phase 2/6 again.
- Why doesn't upstream DNSSEC matter here, and how is this a different boundary than Phase 6's? Phase 6 showed that validation happening at the resolver does not protect an unauthenticated last mile. Phase 7 adds authentication and encryption to that same last mile, and the attacks still work on the permissive client anyway — not because the new protection failed, but because it was never actually engaged for that query once the fallback fired. A protection a client can be talked out of using is not protecting anything.
- DoT vs. DoH? This deployment uses DoT: simpler to reason about and deploy (a dedicated port, no HTTP semantics, BIND's native `tls`/`listen-on tls` support covers it directly), but trivially distinguishable and blockable on its own fixed port — exactly what attempt (iii) exploits. DoH's main advantage is the opposite of that weakness: sharing port 443 with ordinary HTTPS traffic makes it far harder to selectively block without also blocking the web, at the cost of needing an HTTP/2 stack and being one layer more complex to deploy and operate.
- What does an encrypted, authenticated last mile still not protect? The fallback decision itself. It protects the bytes on the wire when the encrypted channel is actually used, but does nothing to stop a client from deciding, on its own, to stop using it — the same failure mode Phase 6 demonstrated at the DNSSEC layer shows up again here one layer down, because the client's willingness to degrade was never something cryptography could cover in the first place.

Generative AI usage

- Phase 2: debugging a cache-poisoning interceptor's checksum-offload bug (forged/relayed packets silently dropped by the receiving kernel despite `pcap_inject` reporting success) and a stale-ARP failure after container restarts, both diagnosed from pasted terminal output and fixed with a background MAC-refresher thread.
- Phase 3: reviewing a DNSSEC signing and chain-of-trust implementation for correctness against the assignment spec.
- Phase 4: implementing signature-stripping (RRSIG/DNSKEY/DS removal with full name decompression to avoid corrupting compression pointers, and RR-count/length recomputation) from scratch in C++, and designing the Run A/Run B signing variants to isolate the DS-omission downgrade.
- Phase 5: reviewing an NSEC zone-walking implementation written with help from another Claude session.
- Phase 6: implementing the relocated last-mile forger and the locally-validating Client B configuration from scratch.
- Phase 7: implementing the DoT resolver configuration, private CA/PKI generation, both client transport postures, and the attacker's tamper/deny/impersonation tooling from scratch, including resolving a BIND `listen-on`/`tls` directive-ordering bug, a TLS private-key permission

bug, and ultimately redesigning the impersonation attempt (ii) after two increasingly complex on-path TCP interception designs — bidirectional diversion, then DNAT — both hit real, topology-specific routing-loop and hairpin-NAT conntrack issues not worth chasing further for what they would actually prove.